Universidad Argentina de la Empresa (UADE)
Ingeniería de Datos I
Segundo cuatrimestre 2026

Trabajo práctico integrador

Base de datos de Red Bull Racing

Gestión interna de una escudería de Fórmula 1 en SQL Server

Docente: Alejandro Feichubuinm

Integrantes

Joaquin Tiago Castellano Acuna
Juan Zeljkovich
Juan Pedro Sama
Santiago Ohoka

Noviembre de 2026

Índice

1. Integrantes 4
2. Descripción del sistema y alcance 5
Descripción del sistema 5
Contexto 5
Objetivo de la base de datos 5
Actores 6
Información a guardar 6
Reglas de negocio 7
Procesos principales 8
Restricciones del sistema 8
Relaciones entre los elementos 8
3. DER 10
Cómo se lee 10
Modelo general 11
Por área 12
Cardinalidades 15
4. Normalización 17
Planilla 1: un fin de semana de carrera 17
Planilla 2: las piezas por temporada 19
Planilla 3: los gastos 19
FNBC 20
Rissanen: que no se pierda nada al separar 20
Excepciones a propósito 21
De dónde sale cada tabla 22
5. Modelo de tablas y diccionario de datos 23
Modelo de tablas 23
Diccionario de datos 24
6. Consultas y subconsultas 32
Qué tiene cada una 32
Consultas 32
Subconsultas 36
7. Vistas, procedimientos, triggers, cursor y funciones 40
Vistas 40

Procedimientos 42
Triggers 44
Cursor 46
Funciones 47
8. Manejo de errores, seguridad y centralización 50
Manejo de errores 50
Seguridad 51
Centralización en la capa de datos 54
9. Conclusiones 56
Las decisiones que más pesaron 56
Qué cambiaríamos 56
Qué aprendimos 57
Anexo. Los scripts 58
Cómo se corren 58
TP_completo.sql 58

1. Integrantes

Cada área la hizo un integrante: sus tablas, sus datos de prueba, sus objetos, sus consultas y su parte del documento. El alcance, el DER y la normalización los hicimos entre todos.

Área	Integrante	Tablas	Objetos, consultas y subconsultas
Deportiva	Juan Pedro Sama	Pais, Temporada, Circuito, GranPremio, TipoSesion, Sesion, Puntaje, Resultado	`fn_puntos_piloto`, `vw_puntos_equipo_por_gp`, `usp_cargar_resultado`, `tr_Sesion_validar`. Consultas 1 a 3 y subconsultas 1 y 2
Personal	Santiago Ohoka	Puesto, Empleado, Piloto, Contrato, AuditoriaContrato	`fn_contrato_vigente`, `vw_nomina`, `usp_inscribir_piloto`, `tr_Contrato`. Consultas 4 a 6 y subconsultas 3 y 4
Técnica	Joaquin Tiago Castellano Acuna	Chasis, Inscripcion, TipoComponente, LimiteComponente, Componente, UsoComponente, Penalizacion	`fn_km_restantes`, `fn_piezas_piloto`, `vw_estado_piezas`, `usp_cerrar_gp` (con el cursor), `usp_retirar_componente`, `tr_UsoComponente`. Consultas 7 a 9 y subconsultas 5 y 6
Finanzas y seguridad	Juan Zeljkovich	CategoriaGasto, Proveedor, Gasto, TopePresupuestario, AlertaTope	`fn_gasto_computable`, `vw_situacion_tope`, `usp_registrar_compra`, `usp_registrar_gasto`, `tr_Gasto_tope`, logins, roles y permisos. Consultas 10 a 12 y subconsultas 7 y 8

El integrador, que juntó los scripts y los corrió en orden, fue Joaquin Tiago Castellano Acuna.

2. Descripción del sistema y alcance

Descripción del sistema

Es una base de datos para la gestión interna de una escudería de Fórmula 1. Elegimos Red Bull Racing. Registra lo que pasa en cada Gran Premio, quién trabaja en el equipo y con qué contrato, qué piezas usa cada auto y cuánto gasta la escudería en el año. Ademas de guardar esos datos hace cumplir las reglas del reglamento y del equipo, como el límite de piezas por temporada o el tope de gasto anual.

Contexto

Una escudería de F1 corre unos 24 Grandes Premios por temporada con dos autos. Cada Gran Premio se corre en un fin de semana con varias sesiones: prácticas, clasificación y carrera, y en algunos fines de semana además una clasificación sprint y una carrera sprint. Las sesiones que reparten puntos lo hacen según la posición de llegada.

Para correr, la escudería tiene que resolver tres cosas además de las carreras.

La primera es el personal. Hay pilotos, ingenieros, mecánicos y administrativos, todos con contrato. De los pilotos, dos son titulares y suele haber uno de reserva.

La segunda son las piezas. El reglamento limita cuántas piezas de cada tipo (motor, caja de cambios, etc.) puede usar cada piloto en la temporada. Si se pasa, recibe una penalización en la grilla de largada. Ademas cada pieza tiene una vida útil en kilómetros, y cuando llega a ese número hay que retirarla.

La tercera es la plata. El reglamento fija un tope de gasto anual. Superarlo trae sanciones graves, así que el equipo necesita saber en todo momento cuánto gastó y cuánto le queda.

Hoy todo eso puede estar repartido en planillas sueltas, y en una planilla nada impide cargar a un piloto sin contrato, usar una pieza que ya se retiró o pasarse del tope sin que nadie lo note.

Objetivo de la base de datos

Centralizar la información deportiva, de personal, técnica y financiera de la escudería, y hacer que la propia base de datos haga cumplir las reglas del reglamento y del equipo. Las reglas viven en la base (restricciones, triggers, procedimientos y permisos) y no en una aplicación. Asi valen para cualquiera que use los datos, por el camino que sea.

Actores

Cada actor despues se convierte en un rol de seguridad.

Actor	Qué hace en el sistema
Director deportivo	Arma el calendario de sesiones, inscribe a los pilotos en cada Gran Premio y carga los resultados
Ingeniero de pista	Registra qué piezas usa cada auto en cada Gran Premio y cierra el Gran Premio cuando termina
Recursos Humanos	Da de alta empleados y contratos y actualiza sueldos. Es el único que ve los sueldos
Finanzas	Registra las compras y los gastos y sigue el tope anual
Consulta	Ve información general (calendario, resultados, nómina sin sueldos), sin modificar nada

Los datos fijos (países, circuitos, tipos de sesión, tabla de puntos, tipos de pieza, límites por temporada y tope de cada año) los carga el administrador de la base.

Información a guardar

Parte deportiva:

- Las temporadas y, para cada una, el calendario de Grandes Premios con su número de fecha, el circuito y las fechas del fin de semana. También si ese fin de semana tiene formato sprint.
- Los circuitos, con su ciudad, su país y su longitud.
- Las sesiones de cada Gran Premio, de qué tipo son y cuándo se corren.
- Cuántos puntos da cada posición en cada tipo de sesión. Esto cambia según la temporada
- El resultado de cada piloto en cada sesión: si terminó, en qué posición; si no, por qué (abandono, descalificación o no largó).

Personal:

- Los empleados, con sus datos personales, su nacionalidad y su puesto.
- De los pilotos, además, su número de auto.
- Los contratos de cada empleado: desde cuándo, hasta cuándo (puede no tener fecha de fin), el sueldo mensual y, para los pilotos, si es titular o reserva.
- El historial de cambios de sueldo: cuándo cambió, de cuánto a cuánto y quién lo hizo.

Parte tecnica:

- Los chasis de cada temporada
- Qué piloto corre cada Gran Premio y con qué chasis.
- Los tipos de pieza con su vida útil en kilómetros, y cuántas piezas de cada tipo se permiten por piloto en cada temporada.
- Cada pieza con su número de serie, su tipo, los kilómetros que lleva y su estado (nueva, en uso, retirada o dañada).
- Qué piezas usó cada piloto en cada Gran Premio y cuántos kilómetros hizo con cada una.
- Las penalizaciones de grilla que recibió cada piloto, por qué pieza y de cuántos puestos.

Finanzas:

- Los proveedores, con su identificación fiscal y su país.
- Las categorías de gasto y si cada una cuenta o no para el tope.

- Cada gasto con su fecha, su monto en dólares, su categoría, su proveedor y, si fue la compra de una pieza, cuál.
- El tope de gasto de cada año y el porcentaje a partir del cual hay que dar aviso.
- Las alertas: qué gasto hizo cruzar ese porcentaje y cuánto se llevaba gastado en ese momento.

Reglas de negocio

Las reglas están numeradas porque cada objeto de la base dice qué regla implementa. La ultima columna es la parte de la base que la hace cumplir.

Regla	Qué dice	Mecanismo
RN01	Las sesiones sprint solo existen en Grandes Premios con formato sprint	Trigger de Sesion
RN02	La fecha de una sesión está dentro del fin de semana de su Gran Premio	Trigger de Sesion
RN03	Un Gran Premio tiene como mucho dos pilotos inscriptos	Procedimiento de inscripción
RN04	Solo se inscribe a un piloto con contrato vigente en la fecha del Gran Premio	Procedimiento de inscripción y función de contrato vigente
RN05	Un chasis no se asigna a dos pilotos en el mismo Gran Premio	UNIQUE en Inscripcion
RN06	El chasis inscripto es de la misma temporada que el Gran Premio	Procedimiento de inscripción
RN07	Un empleado no tiene dos contratos con fechas superpuestas	Trigger de Contrato
RN08	Solo los contratos de pilotos dicen si el piloto es titular o reserva	Trigger de Contrato
RN09	Todo cambio de sueldo queda registrado con fecha y usuario	Trigger de Contrato
RN10	Un resultado solo se carga para un piloto inscripto en el Gran Premio de esa sesión	Procedimiento de carga de resultados
RN11	Solo los que terminan tienen posición, y no hay dos con la misma posición en una sesión	CHECK en Resultado y procedimiento de carga de resultados
RN12	Los puntos dependen de la temporada, el tipo de sesión y la posición, y no se guardan en el resultado	Tabla Puntaje y función de puntos
RN13	Si un piloto usa más piezas de un tipo que las permitidas en la temporada, recibe una penalización: 10 puestos la primera vez y 5 por cada pieza siguiente	Trigger de UsoComponente
RN14	Al cerrar un Gran Premio se suman los kilómetros a cada pieza usada, y la pieza que llega a su vida útil se retira	Procedimiento de cierre con cursor
RN15	Un Gran Premio se cierra una sola vez, y cerrado no acepta más usos de piezas	Procedimiento de cierre y trigger de UsoComponente
RN16	Una pieza retirada o dañada no se puede volver a usar	Trigger de UsoComponente
RN17	Una pieza tiene como mucho un gasto de compra	Trigger de Gasto
RN18	Para el tope solo cuentan los gastos de las categorías marcadas	Función de gasto computable
RN19	Se rechaza un gasto que haría pasar el tope del año, y también uno de un año sin tope cargado	Trigger de Gasto
RN20	Un gasto que hace cruzar el porcentaje de aviso queda registrado como alerta	Trigger de Gasto
RN21	Los sueldos solo los ve Recursos Humanos	Vista de nómina y permisos
RN22	Resultado, Inscripcion, Gasto y Componente solo se modifican mediante sus procedimientos	Permisos

Procesos principales

1. Armar la temporada. El administrador carga la temporada, su tope, sus límites de piezas y su tabla de puntos. El director deportivo carga los Grandes Premios y sus sesiones.
2. Inscribir a los pilotos. Para cada Gran Premio, el director deportivo inscribe a dos pilotos con contrato vigente, cada uno con un chasis de esa temporada.
3. Correr el fin de semana. El ingeniero de pista registra qué piezas usa cada auto y cuántos kilómetros hace con cada una. Si una pieza nueva pasa el límite de la temporada, el piloto queda penalizado. El director deportivo carga los resultados de cada sesión.
4. Cerrar el Gran Premio. El ingeniero de pista lo cierra: a cada pieza usada se le suman los kilómetros, y las que llegaron a su vida útil se retiran. Después del cierre no se pueden cargar más usos de piezas.
5. Comprar. Finanzas registra cada compra. Si es una pieza, la pieza y su gasto se dan de alta juntos. Si el gasto haría pasar el tope, se rechaza. Si hace cruzar el porcentaje de aviso, queda una alerta.
6. Contratar. Recursos Humanos da de alta empleados y contratos y actualiza sueldos. Cada cambio de sueldo queda registrado.

Restricciones del sistema

Son los límites que le pusimos al modelo a propósito.

- Es una sola escudería (Red Bull Racing). No guardamos a los equipos rivales, así que no hay tabla de posiciones del campeonato. Lo que sí se puede calcular es cuántos puntos sumó cada uno de nuestros pilotos.
- Todos los montos estan en dólares y no manejamos tipo de cambio.
- El tope se controla sobre los gastos operativos. Los sueldos no entran en el cálculo (en la F1 real la mayoría sí cuenta).
- El uso de piezas se registra por Gran Premio, no por sesión.
- Una temporada coincide con un año calendario.
- Un gasto de un año que no tiene tope cargado se rechaza.

Relaciones entre los elementos

Cada Gran Premio pertenece a una temporada y se corre en un circuito, y cada circuito está en un país. Un Gran Premio tiene varias sesiones, y cada sesión es de un tipo. La tabla de puntos se define por temporada y tipo de sesión.

Un piloto es un empleado con datos propios. Cada empleado ocupa un puesto, tiene una nacionalidad y firma uno o más contratos a lo largo del tiempo. Cada cambio de sueldo de un contrato queda en su historial.

Un piloto se inscribe en un Gran Premio con un chasis de esa temporada. Esa inscripción es la que después tiene resultados en cada sesión, usos de piezas y penalizaciones. Cada pieza es de un tipo, y cada tipo tiene un límite de piezas por temporada.

Cada gasto es de una categoria y de un proveedor, y puede corresponder a la compra de una pieza. Cada temporada tiene un tope de gasto, y un gasto puede disparar una alerta.

3. DER

El DER está hecho en draw.io: `doc/der/der.drawio`. Tiene cinco pestañas. La primera es el modelo entero sin atributos, para ver cómo se conecta todo, y las otras cuatro son una por área con todos los atributos.

Usamos la notación de Chen que vimos en clase.

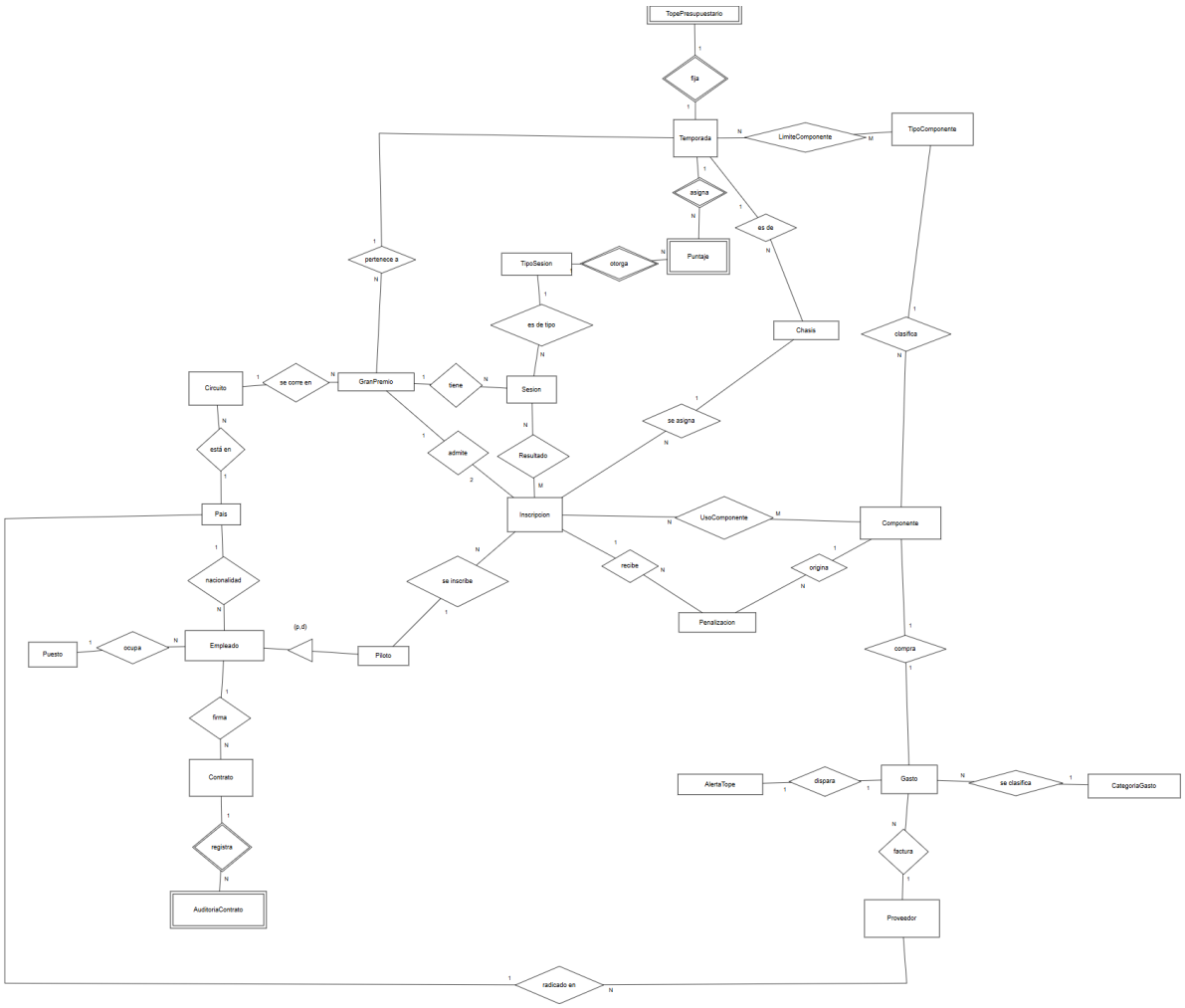
Cómo se lee

- Rectángulo: entidad. Rectángulo doble: entidad débil.
- Rombo: relación. Rombo doble: relación identificadora (la que le da la identidad a una entidad débil).
- Elipse: atributo. Si está subrayado es el identificador de la entidad. Subrayado punteado es la clave parcial de una entidad débil.
- Elipse punteada: atributo derivado, o sea que se puede calcular a partir de otros datos.
- El triángulo es una especialización ("es un").
- En las páginas por área, las cajas grises punteadas son entidades de otra área. Están para que se vea con quién se conecta, sus atributos aparecen en su propia página.

Las cardinalidades van a cada lado del rombo, con 1, N o M. Se leen cruzando: en "GranPremio 1 - tiene - N Sesion" un Gran Premio tiene N sesiones y cada sesión es de 1 Gran Premio. Si los dos lados son muchos se pone N y M.

En "admite" pusimos 2 en vez de N porque un Gran Premio admite como mucho dos inscripciones (los dos autos del equipo).

Modelo general

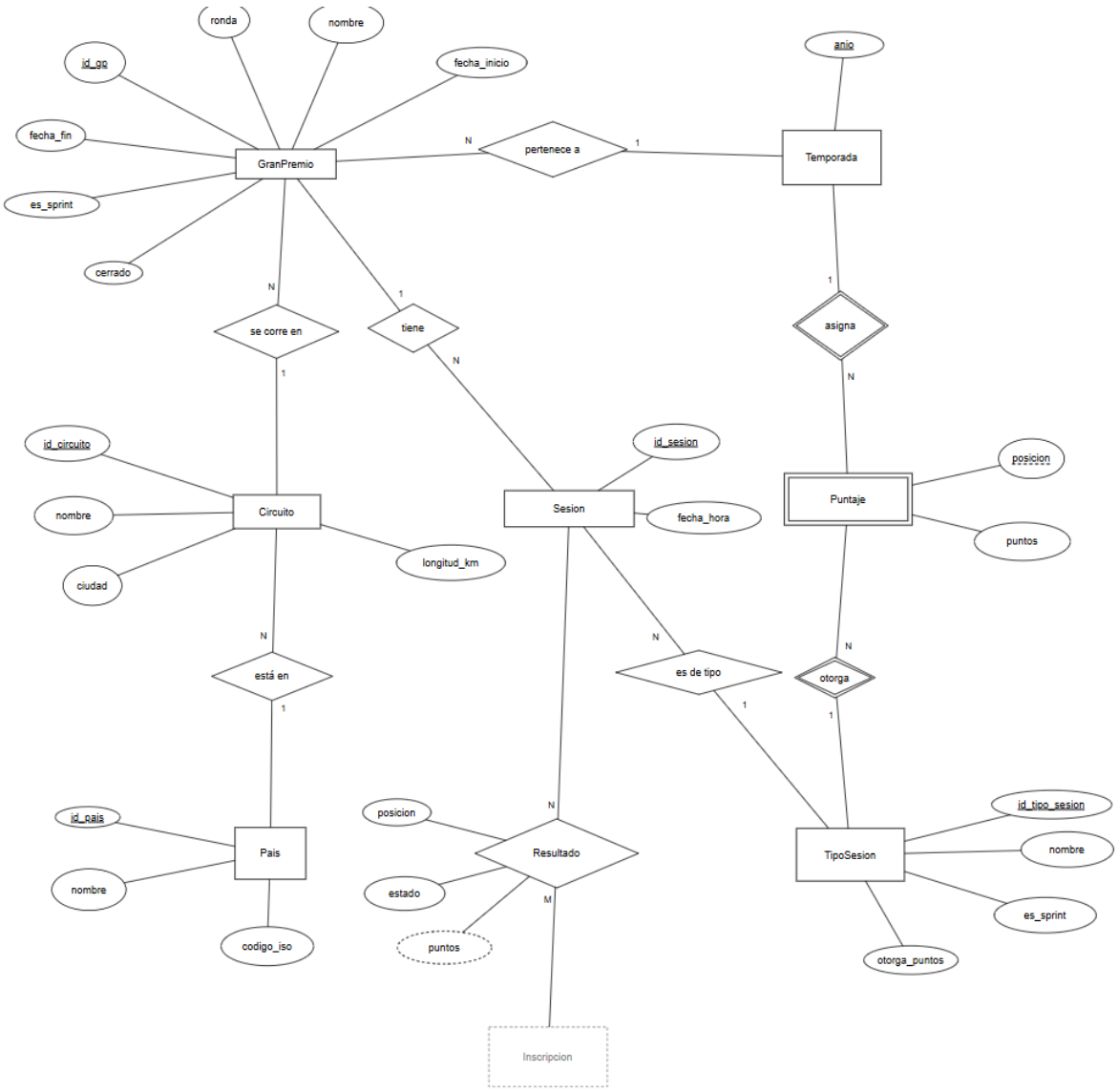


DER general

Son 22 entidades y tres relaciones N:M que tienen atributos propios (Resultado, LimiteComponente y UsoComponente). Cuando se pasa a tablas eso da las 25 tablas del modelo.

Por área

Deportiva



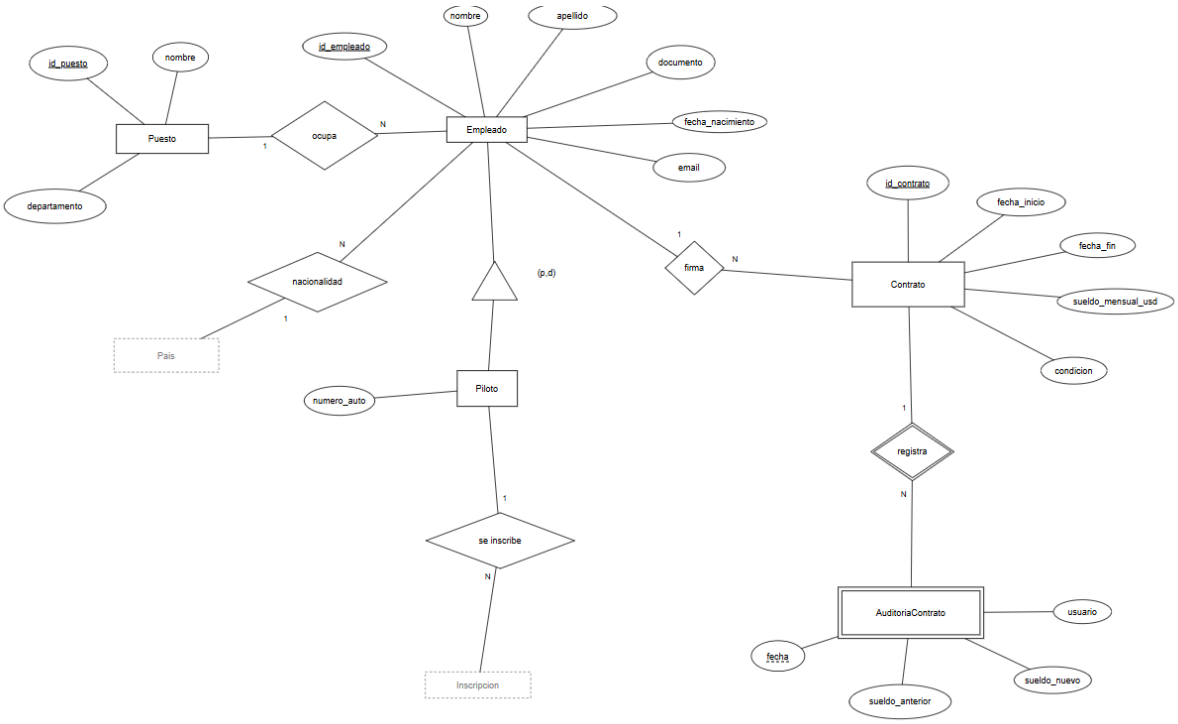
DER deportiva

Temporada se identifica por el año, no le inventamos un id.

Puntaje es débil. Un puntaje solo existe para una temporada y un tipo de sesión, y se identifica por esos dos más la posición (su clave parcial). La tabla de puntos cambia según la temporada y según si es carrera o sprint, por eso depende de las dos.

Resultado es una relación N:M entre Sesion e Inscripcion: en una sesión hay resultados de varios pilotos inscriptos, y un piloto inscripto tiene resultado en varias sesiones del fin de semana. Tiene sus propios atributos, posicion y estado (Finalizó, Abandono, Descalificado, No largó). `puntos` está como derivado porque no lo guardamos, sale de cruzar la posición con Puntaje.

Personal



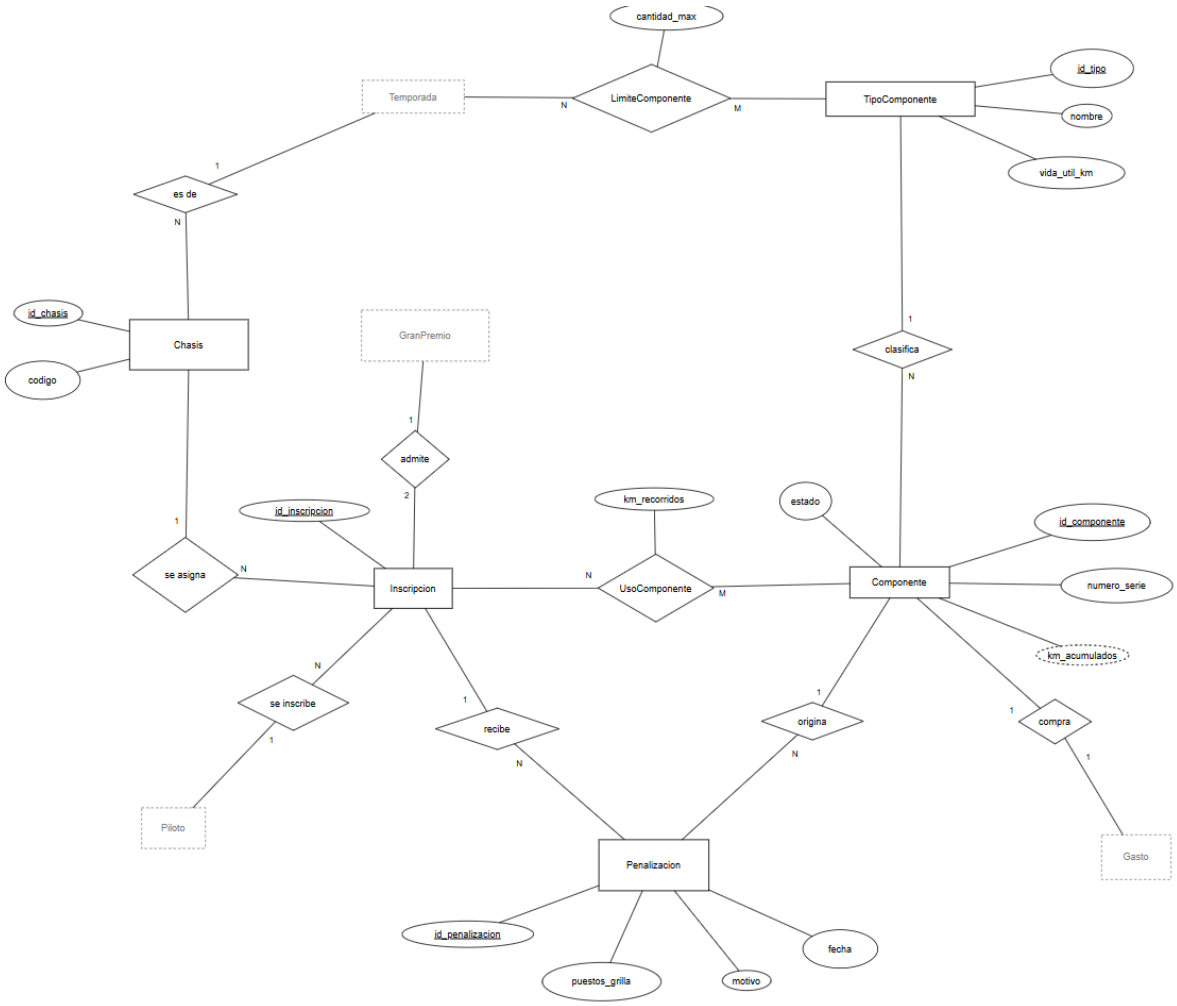
DER personal

Piloto es una especialización de Empleado. Un piloto es un empleado con un dato más, el número de auto. La marcamos (p,d):

- Parcial, porque hay empleados que no son pilotos (ingenieros, mecánicos, administrativos).
- Disjunta, porque solo hay un subtipo y no se puede ser piloto "dos veces".

AuditoriaContrato es débil, depende de Contrato. Un registro de auditoría no tiene sentido sin el contrato al que le cambiaron el sueldo. Su clave parcial es la fecha y hora del cambio. En la tabla igual le pusimos un `id_auditoria` propio, porque dos cambios en el mismo segundo romperían la clave y un id es más simple de manejar.

Técnica



DER técnica

Inscripcion es la entidad que más se conecta. Salió de una relación N:M entre Piloto y GranPremio (un piloto corre muchos GP, un GP lo corren dos pilotos), pero además de unir a los dos lleva el chasis, y de ella cuelgan los resultados, el uso de piezas y las penalizaciones. Por eso la dibujamos como entidad y no como rombo.

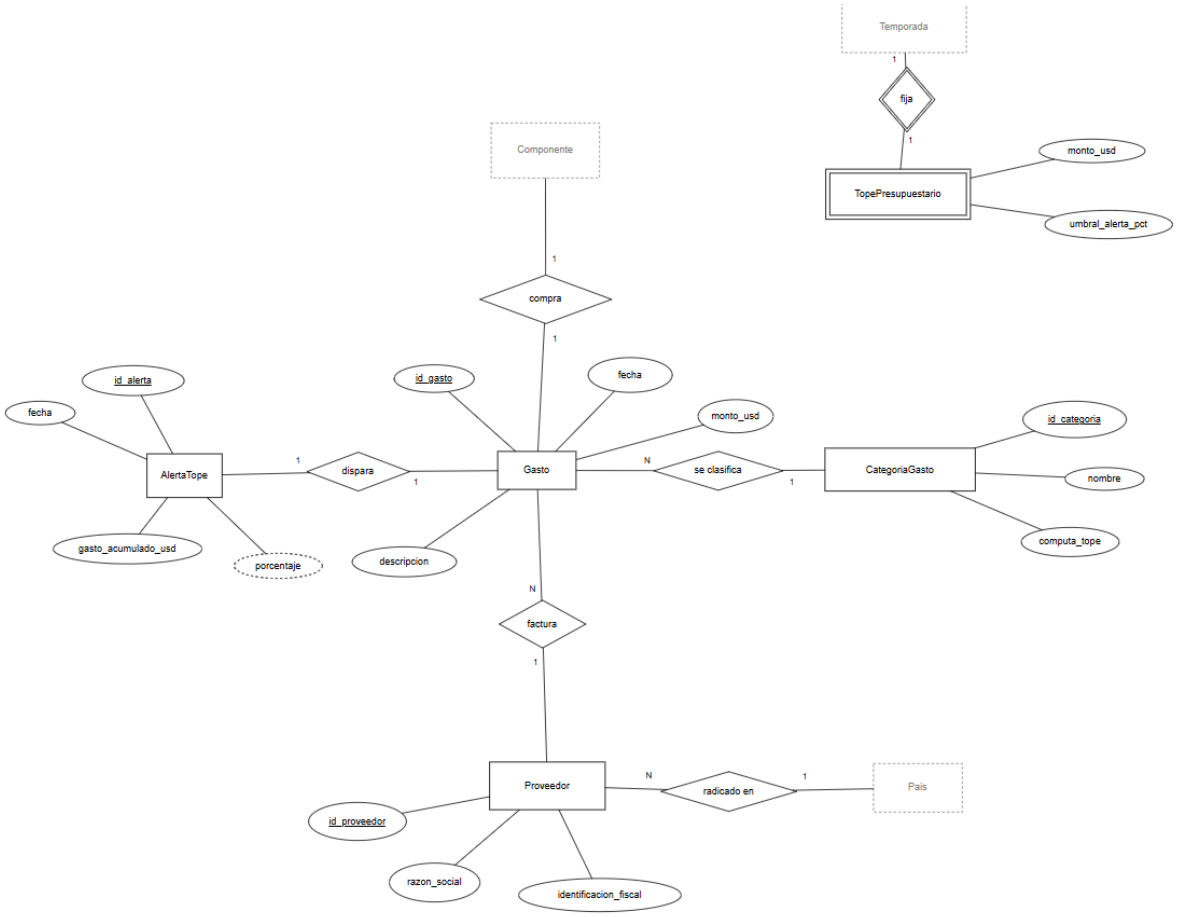
El 2 en "admite" es la regla de que corren dos autos por equipo (RN03).

LimiteComponente es N:M entre Temporada y TipoComponente con el atributo cantidad_max: cuántas piezas de ese tipo se permiten en esa temporada.

UsoComponente es N:M entre Componente e Inscripcion. Guarda los km que hizo cada pieza en cada Gran Premio.

`km_acumulados` de Componente está punteado porque es derivado: es la suma de los km de UsoComponente. Igual lo guardamos en la tabla, es una desnormalización hecha a propósito y está explicada en la sección de normalización.

Finanzas



DER finanzas

TopePresupuestario es débil respecto de Temporada. Hay un tope por año, y el tope se identifica por el año de su temporada, no tiene identificador propio.

La relación "compra" entre Gasto y Componente es 1 a 1 y opcional de los dos lados. No todo gasto es una pieza (hay fletes, servicios, etc.) y una pieza puede no tener gasto cargado, por ejemplo si vino de una temporada anterior. Lo que no puede pasar es que una pieza tenga dos gastos de compra (RN17).

En AlertaTope el porcentaje es derivado (gasto acumulado sobre el tope del año), así que no lo guardamos. Se calcula en la vista.

Cardinalidades

Relación	Entidad	Entidad	Tipo
está en	Pais	Circuito	1:N
se corre en	Circuito	GranPremio	1:N
pertenece a	Temporada	GranPremio	1:N
tiene	GranPremio	Sesion	1:N
es de tipo	TipoSesion	Sesion	1:N
asigna	Temporada	Puntaje	1:N
otorga	TipoSesion	Puntaje	1:N

Relación	Entidad	Entidad	Tipo
Resultado	Sesion	Inscripcion	N:M
nacionalidad	Pais	Empleado	1:N
ocupa	Puesto	Empleado	1:N
firma	Empleado	Contrato	1:N
registra	Contrato	AuditoriaContrato	1:N
se inscribe	Piloto	Inscripcion	1:N
admite	GranPremio	Inscripcion	1:2
es de	Temporada	Chasis	1:N
se asigna	Chasis	Inscripcion	1:N
LimiteComponente	Temporada	TipoComponente	N:M
clasifica	TipoComponente	Componente	1:N
UsoComponente	Inscripcion	Componente	N:M
recibe	Inscripcion	Penalizacion	1:N
origina	Componente	Penalizacion	1:N
fija	Temporada	TopePresupuestario	1:1
se clasifica	CategoriaGasto	Gasto	1:N
factura	Proveedor	Gasto	1:N
radicado en	Pais	Proveedor	1:N
compra	Componente	Gasto	1:1
dispara	Gasto	AlertaTope	1:1

En las 1:N la primera entidad es la del lado 1. Casi todas son opcionales del lado de muchos a propósito, así se puede cargar un circuito antes de que tenga Grandes Premios, un puesto antes de contratar a alguien, etc.

Hay dos reglas que el DER no alcanza a mostrar y que se controlan en la base:

- Que el resultado y la inscripción sean del mismo Gran Premio (la sesión es de un GP y la inscripción también, tienen que coincidir). Lo valida el procedimiento de carga de resultados.
- Que el chasis sea de la misma temporada que el Gran Premio (RN06). Lo valida el procedimiento de inscripción.

4. Normalización

Para mostrar el proceso partimos de tres planillas que alguien de la escudería podría tener hoy en un Excel, y las llevamos paso a paso hasta 3FN. Después revisamos FNBC y que las descomposiciones no pierdan información (Rissanen).

Repaso rápido de lo que usamos:

- Una dependencia funcional A → B quiere decir que si conocemos A conocemos B. Por ejemplo Circuito → País, cada circuito está en un solo país.
- 1FN: cada celda tiene un solo valor y no hay grupos repetidos.
- 2FN: está en 1FN y ningún atributo que no es clave depende de una parte de la clave. Solo puede fallar con claves compuestas.
- 3FN: está en 2FN y ningún atributo que no es clave depende de otro atributo que tampoco es clave (dependencia transitiva).

Planilla 1: un fin de semana de carrera

Así la tendría alguien que anota los resultados del equipo:

Gran Premio	Fecha	Circuito	País	Sesiones	Piloto	N.º auto	Posiciones
GP de Mónaco 2026	2026-06-07	Montecarlo	Mónaco	FP1, FP2, FP3, Clasificación, Carrera	Verstappen	3	5, 7, 4, 6, 6
GP de Mónaco 2026	2026-06-07	Montecarlo	Mónaco	FP1, FP2, FP3, Clasificación, Carrera	Hadjar	6	3, 2, 2, 1, 2

No está en 1FN. Sesiones y Posiciones tienen varios valores en la misma celda, y la posición de cada sesión depende del orden en que se escribió, cosa que nadie controla.

1FN

Armamos una fila por Gran Premio, sesión y piloto. De paso agregamos la temporada, la hora de cada sesión, el estado (si terminó o no) y los puntos, que hoy alguien calcula a mano.

Gran Premio	Temporada	Fecha	Circuito	País	Sesión	Hora	Piloto	N.º auto	Posición	Estado	Puntos
GP de Mónaco 2026	2026	2026-06-07	Montecarlo	Mónaco	Carrera	15:00	Verstappen	3	6	Finalizó	8
GP de Mónaco 2026	2026	2026-06-07	Montecarlo	Mónaco	Carrera	15:00	Hadjar	6	2	Finalizó	18
GP de Mónaco 2026	2026	2026-06-07	Montecarlo	Mónaco	Clasificación	16:00	Verstappen	3	6	Finalizó	0

Clave: (Gran Premio, Sesión, Piloto).

Dependencias funcionales:

- Gran Premio → Temporada, Fecha, Circuito

- Circuito → País
- (Gran Premio, Sesión) → Hora
- Piloto → N.º auto
- (Gran Premio, Sesión, Piloto) → Posición, Estado
- (Temporada, Sesión, Posición) → Puntos

La última es la importante. Los puntos no dependen de quién corrió sino de la posición, del tipo de sesión y del reglamento de ese año.

Con la planilla así aparecen las tres anomalías:

- Modificación: si cambia el país de un circuito o el número de un piloto, hay que corregirlo en cientos de filas. Si se nos pasa una, quedan datos contradictorios.
- Inserción: no se puede cargar un circuito nuevo hasta que se corra una carrera ahí, ni un piloto antes de su primer resultado.
- Borrado: si borramos el único Gran Premio que se corrió en un circuito, perdemos el circuito y su país.

Y una cuarta que es propia de este caso: si la FIA cambia la tabla de puntos, hay que recalcular a mano todos los resultados de la temporada.

2FN

Sacamos lo que depende de una parte de la clave:

- GranPremio (**Gran Premio**, Temporada, Fecha, Circuito, País), por Gran Premio → ...
- Sesion (**Gran Premio, Sesión**, Hora), por (Gran Premio, Sesión) → Hora
- Piloto (**Piloto**, N.º auto), por Piloto → N.º auto
- Resultado (**Gran Premio, Sesión, Piloto**, Posición, Estado, Puntos)

3FN

Quedan dos dependencias transitivas:

- En GranPremio, País depende de Circuito, que no es clave. Sale Circuito (**Circuito**, País) y GranPremio se queda con Circuito como referencia.
- En Resultado, Puntos depende de (Temporada, Sesión, Posición). La temporada sale del Gran Premio, así que Puntos depende de la posición y no de la clave entera. Sale Puntaje (**Temporada, Sesión, Posición**, Puntos) y Resultado ya no guarda los puntos.

Resultado sin los puntos queda (**Gran Premio, Sesión, Piloto**, Posición, Estado). Los puntos se calculan cuando hacen falta, juntando Resultado con su Gran Premio (para saber la temporada) y con Puntaje. Eso hace la función `fn_puntos_piloto`.

Antes de pasar a tablas hicimos tres cambios más, que no son de normalización sino de diseño:

1. "Clasificación", "Carrera", "Sprint" se repetían como texto. Pasaron a un catálogo, TipoSesion, que además dice si la sesión es sprint y si da puntos.
2. El par (Gran Premio, Piloto) se convirtió en Inscripcion, porque la parte técnica necesita saber con qué chasis corrió cada piloto en cada GP. Resultado apunta a la inscripción en vez de al par.
3. Reemplazamos los nombres por claves numéricas (`id_gp`, `id_circuito`, ...).

Planilla 2: las piezas por temporada

Tipo de pieza	Temporada	Vida útil (km)	Máximo por temporada
Motor	2026	7000	4
Motor	2027	7000	3
Caja de cambios	2026	9000	5

Está en 1FN. La clave es (Tipo, Temporada).

Dependencias:

- (Tipo, Temporada) → Máximo, porque el reglamento cambia el límite de un año a otro
- Tipo → Vida útil

La vida útil depende solo del tipo, o sea de una parte de la clave. Es una dependencia parcial y la planilla no está en 2FN. Se ve en la tabla: el 7000 del motor está repetido, y si en algún momento cambia hay que tocarlo en todas las temporadas. Tampoco se puede cargar un tipo de pieza nuevo hasta saber su límite para algún año.

Separamos:

- TipoComponente (**Tipo**, Vida útil)
- LimiteComponente (**Tipo, Temporada**, Máximo)

Las dos quedan en 3FN porque no tienen atributos no clave que dependan entre sí. Es el ejemplo más claro de 2FN que tenemos.

Planilla 3: los gastos

N.º gasto	Fecha	Monto (USD)	Proveedor	Identificación fiscal	País del proveedor	Categoría	¿Cuenta para el tope?
1	2026-03-02	120000	Brembo	IT001	Italia	Fabricación de piezas	Sí
2	2026-03-09	45000	Brembo	IT001	Italia	Fabricación de piezas	Sí
3	2026-03-10	8000	Do & Co	AT007	Austria	Hospitalidad y marketing	No

Clave: N.º gasto. Como la clave es simple está en 2FN automáticamente.

Dependencias:

- N.º gasto → Fecha, Monto, Proveedor, Categoría
- Proveedor → Identificación fiscal, País del proveedor
- Identificación fiscal → Proveedor
- Categoría → ¿Cuenta para el tope?

Hay dos dependencias transitivas: los datos del proveedor dependen del proveedor y "¿Cuenta para el tope?" depende de la categoría. Ninguno de los dos es clave de la planilla.

Las anomalías: si un proveedor cambia de país hay que corregir todos sus gastos, no se puede dar de alta un proveedor sin haberle comprado algo, y si alguien marca "No" en un gasto de fabricación de piezas y "Sí" en otro, la planilla no sabe cuál vale.

En 3FN queda:

- Gasto (**N.º gasto**, Fecha, Monto, Proveedor, Categoría)
- Proveedor (**Proveedor**, Identificación fiscal, País)
- CategoriaGasto (**Categoría**, Cuenta para el tope)

Las alertas del tope salen del mismo razonamiento. Al principio AlertaTope tenía año y porcentaje. El año sale de la fecha del gasto (id_gasto → fecha → año), es una transitiva, así que lo sacamos. El porcentaje es gasto acumulado dividido el tope del año, o sea un dato calculado, y se muestra en la vista `vw_situacion_tope`.

FNBC

La definición que vimos en la clase 13: una relación está en FNBC si para cada dependencia funcional no trivial X → Y, X es una superclave.

Revisamos las tablas finales que tienen más de una clave candidata, que es donde puede fallar:

- Inscripcion. Tiene tres claves candidatas: `id_inscripcion`, (`id_gp`, `id_piloto`) y (`id_gp`, `id_chasis`). Todas las dependencias salen de alguna de ellas. Está en FNBC.
- Resultado. La clave es (`id_sesion`, `id_inscripcion`). Además, para los que terminaron, (`id_sesion`, `posicion`) → `id_inscripcion`, porque no puede haber dos pilotos en la misma posición. Eso también es una clave (la controla el procedimiento que carga los resultados), así que sigue en FNBC.
- Puntaje, LimiteComponente, UsoComponente: el único determinante es la clave compuesta. FNBC.
- Proveedor: `id_proveedor` e `identificacion_fiscal` son claves candidatas y se determinan entre sí. FNBC.
- GranPremio: este no está en FNBC y lo dejamos así a propósito, está explicado más abajo.

El resto de las tablas tiene una sola clave y todo depende de ella, así que 3FN y FNBC coinciden.

Rissanen: que no se pierda nada al separar

En la clase vimos que una descomposición de R en R1 y R2 es independiente si y solo si:

1. Toda dependencia funcional de R se puede deducir de las dependencias de R1 y R2, y
2. Los atributos comunes de R1 y R2 forman una clave candidata de al menos una de las dos.

La segunda condición es la que asegura que al hacer el JOIN volvemos a tener exactamente los datos de antes. Lo revisamos en cada separación:

Original	R1	R2	Atributos comunes	¿Clave de cuál?	¿Se conservan las dependencias?
GranPremio (planilla 1)	GranPremio (GP, Temporada, Fecha, Circuito)	Circuito (Circuito, País)	Circuito	de Circuito	Sí. GP → Circuito queda en R1 y Circuito → País en R2
Resultado (planilla 1)	Resultado (GP, Sesión, Piloto, Posición, Estado)	Puntaje (Temporada, Sesión, Posición, Puntos)	Sesión, Posición, y la temporada a través del GP	de Puntaje	Sí, ver abajo
Planilla 2	TipoComponente (Tipo, Vida útil)	LimiteComponente (Tipo, Temporada, Máximo)	Tipo	de TipoComponente	Sí. Tipo → Vida útil en R1 y (Tipo, Temporada) → Máximo en R2
Planilla 3	Gasto (..., Proveedor, Categoría)	Proveedor (Proveedor, Id. fiscal, País)	Proveedor	de Proveedor	Sí
Planilla 3	Gasto (..., Categoría)	CategoriaGasto (Categoría, Cuenta)	Categoría	de CategoriaGasto	Sí

El caso de Resultado y Puntaje es el único que no es directo, porque la temporada no está en Resultado. Si lo separamos primero con el Gran Premio, R2 = (GP, Sesión, Posición, Puntos), los atributos comunes son (GP, Sesión, Posición), que es clave de R2, y cumple Rissanen. Después cambiamos GP por su temporada en R2, porque los puntos son los mismos para todos los Grandes Premios de un año. Como GP → Temporada está en GranPremio, el JOIN Resultado ⋈ GranPremio ⋈ Puntaje devuelve los mismos puntos que tenía la planilla.

Excepciones a propósito

Hay dos lugares donde no seguimos la normalización al pie de la letra. Los dejamos así sabiendo lo que hacemos.

Los km acumulados de cada pieza

Componente guarda `km_acumulados`, aunque se podría calcular sumando los km de esa pieza en UsoComponente. Es una desnormalización: el mismo dato está en dos lugares.

La hacemos porque el estado de las piezas se consulta todo el tiempo, y porque el cierre de cada Gran Premio decide si retira una pieza según esos km. Sumar todo el historial cada vez no tiene sentido.

El riesgo es que el número guardado no coincida con la suma real. Lo controlamos así:

- Solo el procedimiento `usp_cerrar_gp` modifica ese campo, una vez por Gran Premio.
- Ningún rol tiene permiso de UPDATE directo sobre Componente.
- Un Gran Premio cerrado no acepta más usos de piezas (RN15), así que lo que se sumó no cambia después.

En el DER `km_acumulados` aparece como atributo derivado por esto mismo.

El año del Gran Premio

GranPremio tiene `anio` (la temporada) y `fecha_inicio`. Como una temporada es un año calendario, `fecha_inicio` → `anio`, y `fecha_inicio` no es superclave. Por eso la tabla no está en FNBC.

Igual sí está en 3FN. `anio` forma parte de la clave candidata (`anio`, `ronda`), y 3FN solo prohíbe que dependan transitivamente los atributos que no son parte de ninguna clave.

Lo dejamos porque `anio` es la clave foránea a Temporada y es lo que permite decir "la ronda 8 de 2026" con un UNIQUE. Para que el dato repetido no pueda contradecirse le pusimos un CHECK: `YEAR(fecha_inicio) = anio`.

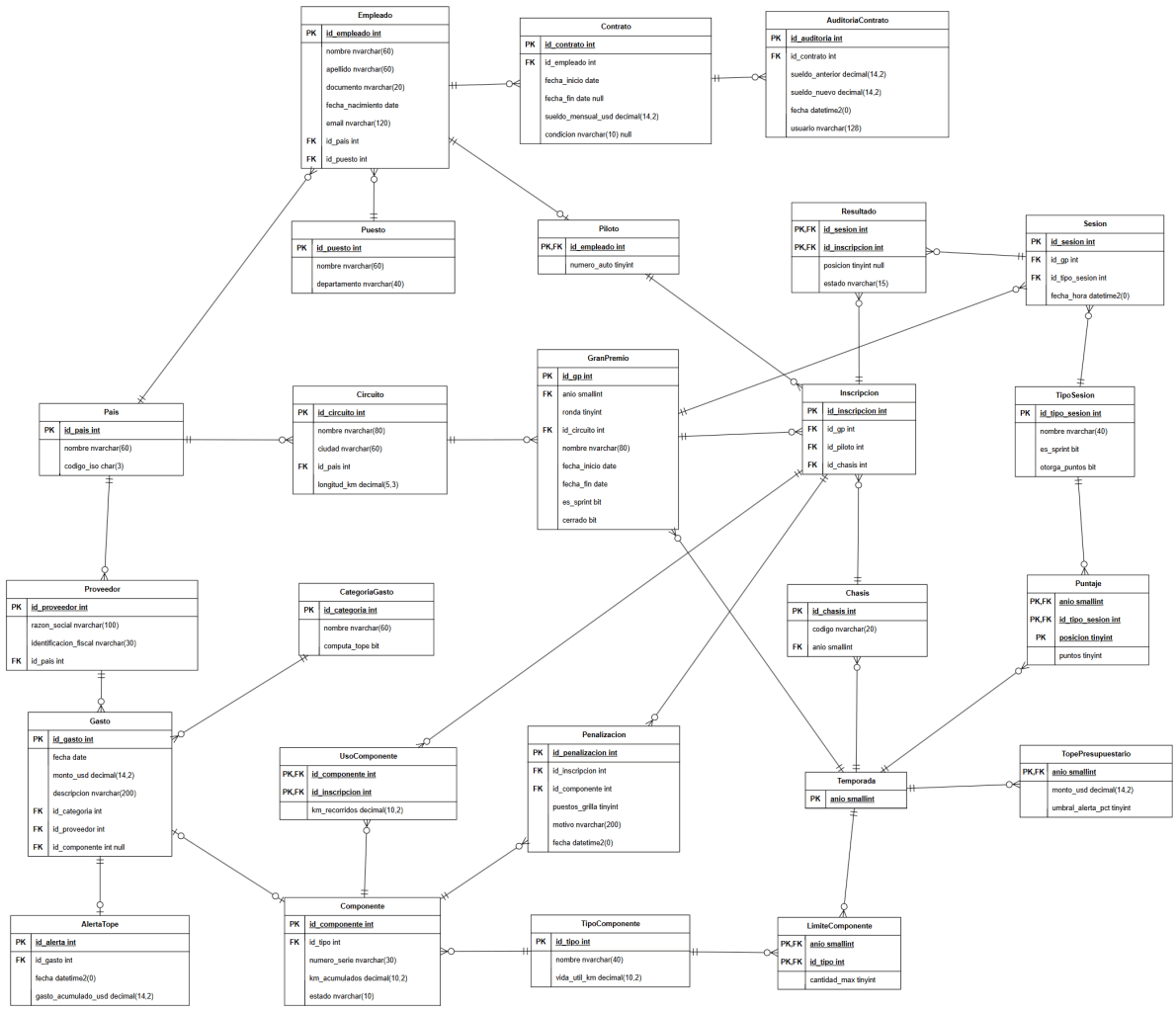
De dónde sale cada tabla

Tabla	Sale de
Pais, Circuito, GranPremio, Temporada	planilla 1 (3FN de la parte del Gran Premio)
Sesion, TipoSesion	planilla 1 (2FN más el catálogo de tipos)
Resultado, Puntaje	planilla 1 (3FN de los resultados)
Inscripcion	planilla 1, el par (Gran Premio, Piloto) con el chasis
Empleado, Piloto	planilla 1 (Piloto → N.º auto), ampliado con los datos de personal
TipoComponente, LimiteComponente	planilla 2
Gasto, Proveedor, CategoriaGasto, AlertaTope	planilla 3
Puesto, Contrato, AuditoriaContrato	no salen de una planilla. Las armamos directo desde el alcance, y ya nacen normalizadas (cada una tiene una clave simple y nada depende de otra cosa que la clave)
Chasis, Componente, UsoComponente, Penalizacion, TopePresupuestario	lo mismo, salen del alcance y del DER

5. Modelo de tablas y diccionario de datos

Modelo de tablas

Son las 25 tablas como quedaron en la base, con sus columnas, tipos y claves. Está en draw.io: `doc/tablas/modelo_tablas.drawio`.



Modelo de tablas

Cómo se lee:

- PK es la clave primaria y FK una clave foránea. Si dice PK,FK la columna es las dos cosas, como en Piloto o en las tablas con clave compuesta.
- Las líneas unen cada clave foránea con la tabla a la que apunta. Del lado de muchos va la pata de gallo, del lado de uno las dos rayitas.
- Tres relaciones son de uno a uno: Piloto con Empleado (la especialización), AlertaTope con Gasto (un gasto dispara como mucho una alerta) y Gasto con Componente (una pieza tiene como mucho un gasto de compra, RN17). Por eso esas tienen un círculo y una raya en vez de pata de gallo.
- `null` al lado del tipo quiere decir que la columna acepta nulos. Son solo cuatro: `fecha_fin` y `condicion` en Contrato, `posicion` en Resultado e `id_componente` en Gasto.

Las restricciones (CHECK, UNIQUE, FK) no entran en el dibujo, están en el diccionario de abajo y en `scripts/01_estructura.sql`. Todas tienen nombre, así que si una falla el mensaje dice cuál fue.

Diccionario de datos

Está ordenado por área. En "Restricción" ponemos lo que controla la propia tabla (claves, UNIQUE, CHECK). Cuando una regla la controla un trigger o un procedimiento, lo aclaramos entre paréntesis con su número de regla.

Deportiva

Pais

Los países. Se usan para la nacionalidad de los empleados, para ubicar los circuitos y para los proveedores.

Columna	Tipo	Nulos	Default	Restricción	Descripción
id_pais	INT IDENTITY	No		Clave primaria	Identificador del país
nombre	NVARCHAR(60)	No		Único	Nombre del país
codigo_iso	CHAR(3)	No		Único; tres letras mayúsculas	Código ISO de tres letras (ARG, GBR, JPN)

Temporada

Una fila por año. Todo lo que cambia de una temporada a otra (el calendario, la tabla de puntos, los chasis, los límites de piezas y el tope) apunta acá.

Columna	Tipo	Nulos	Default	Restricción	Descripción
anio	SMALLINT	No		Clave primaria; desde 1950	Año de la temporada

Circuito

Columna	Tipo	Nulos	Default	Restricción	Descripción
id_circuito	INT IDENTITY	No		Clave primaria	Identificador del circuito
nombre	NVARCHAR(80)	No		Único	Nombre del circuito
ciudad	NVARCHAR(60)	No			Ciudad donde está
id_pais	INT	No		FK a Pais	País del circuito
longitud_km	DECIMAL(5,3)	No		Mayor a 0	Largo de una vuelta, en km

GranPremio

Cada fecha del calendario.

Columna	Tipo	Nulos	Default	Restricción	Descripción
id_gp	INT IDENTITY	No		Clave primaria	Identificador del Gran Premio
anio	SMALLINT	No		FK a Temporada; tiene que ser el año de fecha_inicio	Temporada a la que pertenece
ronda	TINYINT	No		Mayor a 0; única dentro de la temporada	Número de fecha del calendario
id_circuito	INT	No		FK a Circuito	Dónde se corre
nombre	NVARCHAR(80)	No			Nombre del Gran Premio ("GP de Japon")
fecha_inicio	DATE	No			Primer día del fin de semana
fecha_fin	DATE	No		No puede ser anterior a fecha_inicio	Último día del fin de semana
es_sprint	BIT	No	0		1 si el fin de semana tiene formato sprint
cerrado	BIT	No	0	Solo lo cambia `usp_cerrar_gp` (RN15)	1 cuando ya se sumaron los km de las piezas

TipoSesion

Columna	Tipo	Nulos	Default	Restricción	Descripción
id_tipo_sesion	INT IDENTITY	No		Clave primaria	Identificador del tipo de sesión
nombre	NVARCHAR(40)	No		Único	Practica 1, Clasificacion, Sprint, Carrera, etc
es_sprint	BIT	No	0		1 si solo existe en los fines de semana sprint
otorga_puntos	BIT	No	0		1 si reparte puntos (Sprint y Carrera)

Sesion

Columna	Tipo	Nulos	Default	Restricción	Descripción
id_sesion	INT IDENTITY	No		Clave primaria	Identificador de la sesión
id_gp	INT	No		FK a GranPremio	Gran Premio de la sesión
id_tipo_sesion	INT	No		FK a TipoSesion; único junto con id_gp. Si es sprint, el GP tiene que ser sprint (trigger, RN01)	Qué sesión es. Un GP tiene una sola de cada tipo
fecha_hora	DATETIME2(0)	No		Dentro del fin de semana del GP (trigger, RN02)	Cuándo empieza

Puntaje

La tabla de puntos de cada temporada. Las posiciones que no dan puntos no se cargan.

Columna	Tipo	Nulos	Default	Restricción	Descripción
anio	SMALLINT	No		Clave primaria (con las otras dos); FK a Temporada	Temporada
id_tipo_sesion	INT	No		Clave primaria; FK a TipoSesion	Carrera o Sprint
posicion	TINYINT	No		Clave primaria; mayor a 0	Posición de llegada
puntos	TINYINT	No		Mayor a 0	Puntos que da esa posición

Resultado

Cómo le fue a cada piloto en cada sesión. No guarda los puntos (RN12), salen de cruzar con Puntaje.

Columna	Tipo	Nulos	Default	Restricción	Descripción
id_sesion	INT	No		Clave primaria (con id_inscripcion); FK a Sesion	Sesión
id_inscripcion	INT	No		Clave primaria; FK a Inscripcion. Tiene que ser del mismo GP que la sesión (procedimiento, RN10)	Piloto inscripto en ese GP
posicion	TINYINT	Sí		Obligatoria si terminó y vacía si no (CHECK); no se repite en la sesión (procedimiento de carga). RN11	Posición de llegada
estado	NVARCHAR(15)	No		'Finalizo', 'Abandono', 'Descalificado' o 'No largo'	Cómo terminó

Personal

Puesto

Columna	Tipo	Nulos	Default	Restricción	Descripción
id_puesto	INT IDENTITY	No		Clave primaria	Identificador del puesto
nombre	NVARCHAR(60)	No		Único	Piloto, Mecanico, Director de equipo, etc
departamento	NVARCHAR(40)	No			Deportivo, Tecnico, Direccion o Administracion

Empleado

Columna	Tipo	Nulos	Default	Restricción	Descripción
id_empleado	INT IDENTITY	No		Clave primaria	Identificador del empleado
nombre	NVARCHAR(60)	No			Nombre
apellido	NVARCHAR(60)	No			Apellido
documento	NVARCHAR(20)	No		Único	Documento o pasaporte
fecha_nacimiento	DATE	No		Desde 1920-01-01	Fecha de nacimiento
email	NVARCHAR(120)	No		Único; tiene que tener forma de mail	Mail de trabajo
id_pais	INT	No		FK a Pais	Nacionalidad
id_puesto	INT	No		FK a Puesto	Puesto que ocupa

Piloto

Es la especialización de Empleado: un piloto es un empleado con número de auto. Comparte la clave con Empleado.

Columna	Tipo	Nulos	Default	Restricción	Descripción
id_empleado	INT	No		Clave primaria; FK a Empleado	El empleado que es piloto
numero_auto	TINYINT	No		Único; entre 1 y 99	Número del auto

Contrato

Columna	Tipo	Nulos	Default	Restricción	Descripción
id_contrato	INT IDENTITY	No		Clave primaria	Identificador del contrato
id_empleado	INT	No		FK a Empleado. No puede tener otro contrato que se superponga (trigger, RN07)	Empleado
fecha_inicio	DATE	No			Desde cuándo vale
fecha_fin	DATE	Sí		Posterior a fecha_inicio	Hasta cuándo vale. Null si no tiene fecha de fin
sueldo_mensual_usd	DECIMAL(14,2)	No		Mayor a 0. Cada cambio queda en AuditoriaContrato (trigger, RN09)	Sueldo mensual en dólares. Solo lo ve RRHH (RN21)
condicion	NVARCHAR(10)	Sí		'Titular' o 'Reserva'. Va solo en contratos de pilotos, y en esos es obligatoria (trigger, RN08)	Si el piloto es titular o reserva

AuditoriaContrato

El historial de cambios de sueldo. La llena el trigger de Contrato y nadie la puede modificar.

Columna	Tipo	Nulos	Default	Restricción	Descripción
id_auditoria	INT IDENTITY	No		Clave primaria	Identificador del registro
id_contrato	INT	No		FK a Contrato	Contrato que cambió
sueldo_anterior	DECIMAL(14,2)	No			Sueldo antes del cambio
sueldo_nuevo	DECIMAL(14,2)	No			Sueldo después del cambio
fecha	DATETIME2(0)	No	getdate()		Cuándo se hizo el cambio
usuario	NVARCHAR(128)	No	system_user		Login que hizo el cambio

Técnica

Chasis

Columna	Tipo	Nulos	Default	Restricción	Descripción
id_chasis	INT IDENTITY	No		Clave primaria	Identificador del chasis
codigo	NVARCHAR(20)	No		Único	Código del chasis (RB22-01)
anio	SMALLINT	No		FK a Temporada	Temporada para la que se construyó

Inscripcion

Qué piloto corre cada Gran Premio y con qué chasis. Es la tabla a la que se cuelgan los resultados, los usos de piezas y las penalizaciones.

Columna	Tipo	Nulos	Default	Restricción	Descripción
id_inscripcion	INT IDENTITY	No		Clave primaria	Identificador de la inscripción
id_gp	INT	No		FK a GranPremio. Como mucho dos por GP (procedimiento, RN03)	Gran Premio
id_piloto	INT	No		FK a Piloto; único junto con id_gp. Con contrato vigente (procedimiento, RN04)	Piloto. Se llama distinto que la columna a la que apunta (`Piloto.id_empleado`) porque acá cumple el rol de piloto
id_chasis	INT	No		FK a Chasis; único junto con id_gp (RN05). De la misma temporada (procedimiento, RN06)	Chasis con el que corre

TipoComponente

Columna	Tipo	Nulos	Default	Restricción	Descripción
id_tipo	INT IDENTITY	No		Clave primaria	Identificador del tipo
nombre	NVARCHAR(40)	No		Único	Motor de combustion, Turbo, MGU-K, Caja de cambios
vida_util_km	DECIMAL(10,2)	No		Mayor a 0	Km a partir de los cuales la pieza se retira

LimiteComponente

Columna	Tipo	Nulos	Default	Restricción	Descripción
anio	SMALLINT	No		Clave primaria (con id_tipo); FK a Temporada	Temporada
id_tipo	INT	No		Clave primaria; FK a TipoComponente	Tipo de pieza
cantidad_max	TINYINT	No		Mayor a 0	Cuántas piezas de ese tipo puede usar cada piloto en la temporada

Componente

Cada pieza física, con su número de serie.

Columna	Tipo	Nulos	Default	Restricción	Descripción
id_componente	INT IDENTITY	No		Clave primaria	Identificador de la pieza
id_tipo	INT	No		FK a TipoComponente	Tipo de pieza
numero_serie	NVARCHAR(30)	No		Único	Número de serie (GBX-26-01)
km_acumulados	DECIMAL(10,2)	No	0	Mayor o igual a 0. Solo lo cambia `usp_cerrar_gp` (RN14)	Km que lleva la pieza. Es la desnormalización explicada en la normalización
estado	NVARCHAR(10)	No	'Nuevo'	'Nuevo', 'En uso', 'Retirado' o 'Danado'	Estado de la pieza

UsoComponente

Qué piezas usó cada piloto en cada Gran Premio.

Columna	Tipo	Nulos	Default	Restricción	Descripción
id_componente	INT	No		Clave primaria (con id_inscripcion); FK a Componente. No puede estar retirada ni dañada (trigger, RN16)	Pieza
id_inscripcion	INT	No		Clave primaria; FK a Inscripcion. El GP no puede estar cerrado (trigger, RN15)	Piloto y Gran Premio
km_recorridos	DECIMAL(10,2)	No		Mayor o igual a 0	Km que hizo con esa pieza en ese GP

Penalizacion

La llena el trigger de UsoComponente cuando una pieza nueva hace pasar el límite (RN13).

Columna	Tipo	Nulos	Default	Restricción	Descripción
id_penalizacion	INT IDENTITY	No		Clave primaria	Identificador de la penalización
id_inscripcion	INT	No		FK a Inscripcion	Piloto y Gran Premio penalizado
id_componente	INT	No		FK a Componente	Pieza que hizo pasar el límite
puestos_grilla	TINYINT	No		Mayor a 0	10 la primera vez, 5 por cada pieza siguiente
motivo	NVARCHAR(200)	No			Texto armado por el trigger ("4 MGU-K usados en la temporada y el limite es 3")
fecha	DATETIME2(0)	No	getdate()		Cuándo se aplicó

Finanzas

CategoriaGasto

Columna	Tipo	Nulos	Default	Restricción	Descripción
id_categoria	INT IDENTITY	No		Clave primaria	Identificador de la categoría
nombre	NVARCHAR(60)	No		Único	Fabricacion de piezas, Logistica y fletes, etc
computa_tope	BIT	No	1		1 si los gastos de esta categoría cuentan para el tope (RN18)

Proveedor

Columna	Tipo	Nulos	Default	Restricción	Descripción
id_proveedor	INT IDENTITY	No		Clave primaria	Identificador del proveedor
razon_social	NVARCHAR(100)	No			Razón social
identificacion_fiscal	NVARCHAR(30)	No		Único	CUIT, VAT o lo que corresponda al país
id_pais	INT	No		FK a Pais	País del proveedor

Gasto

Columna	Tipo	Nulos	Default	Restricción	Descripción
id_gasto	INT IDENTITY	No		Clave primaria	Identificador del gasto
fecha	DATE	No		Su año tiene que tener tope cargado (trigger, RN19)	Fecha del gasto
monto_usd	DECIMAL(14,2)	No		Mayor a 0. No puede hacer pasar el tope (trigger, RN19)	Monto en dólares
descripcion	NVARCHAR(200)	No			Qué se pagó
id_categoria	INT	No		FK a CategoriaGasto	Categoría
id_proveedor	INT	No		FK a Proveedor	A quién se le pagó
id_componente	INT	Sí		FK a Componente; una pieza tiene un solo gasto de compra (trigger, RN17)	Pieza comprada, si el gasto es la compra de una pieza

TopePresupuestario

Columna	Tipo	Nulos	Default	Restricción	Descripción
anio	SMALLINT	No		Clave primaria; FK a Temporada	Año
monto_usd	DECIMAL(14,2)	No		Mayor a 0	Tope de gasto del año
umbral_alerta_pct	TINYINT	No	90	Entre 1 y 99	Porcentaje del tope a partir del cual se avisa

AlertaTope

La llena el trigger de Gasto (RN20).

Columna	Tipo	Nulos	Default	Restricción	Descripción
id_alerta	INT IDENTITY	No		Clave primaria	Identificador de la alerta
id_gasto	INT	No		FK a Gasto; único	Gasto que hizo cruzar el umbral
fecha	DATETIME2(0)	No	getdate()		Cuándo saltó
gasto_acumulado_usd	DECIMAL(14,2)	No			Gasto computable del año contando ese gasto

6. Consultas y subconsultas

Están en `scripts/08_consultas.sql`. Cada una responde una pregunta que alguien de la escudería se haría de verdad, y están agrupadas por área.

Se corren con la base recién armada (scripts 01 a 07), y los resultados que ponemos son con esos datos. El script de pruebas los cambia (cierra Grandes Premios, carga compras), así que si se corre antes los números no van a ser los mismos.

Qué tiene cada una

Lo que tenía que aparecer	Dónde
SELECT con WHERE y ORDER BY	Consultas 5, 7 y casi todas las demás
JOIN entre tres o más tablas	Consultas 1, 2, 7, 8 y 12
LEFT, RIGHT o FULL JOIN	Consulta 1 (LEFT) y consulta 3 (RIGHT)
GROUP BY con funciones de agregación	Consultas 2, 4, 6, 8, 9, 10 y 11
HAVING	Consultas 2, 8 y 11
Subconsulta con IN o NOT IN	Subconsultas 3 y 7 (NOT IN)
Subconsulta correlacionada o con EXISTS	Subconsulta 4 (EXISTS), y 5, 6 y 8 (correlacionadas)
Subconsulta con ANY o ALL	Subconsultas 3 (ANY) y 6 (ALL)
Subconsulta en el FROM o en el SELECT	Subconsultas 1 (FROM) y 2 (SELECT)

Por área quedan así: Deportiva tiene las consultas 1 a 3 y las subconsultas 1 y 2, Personal las consultas 4 a 6 y las subconsultas 3 y 4, Técnica las consultas 7 a 9 y las subconsultas 5 y 6, y Finanzas las consultas 10 a 12 y las subconsultas 7 y 8.

Consultas

Deportiva

Consulta 1. ¿Qué hizo cada piloto en las carreras y sprints de 2026, y cuántos puntos sacó en cada una?

Son seis tablas con `inner join` y Puntaje con `left join`. Un abandono, o una posición que no da puntos, no tiene fila en Puntaje, y con `inner join` desaparecería del listado. Con `left join` la fila queda y el `isnull` la muestra con 0. Solo trae las sesiones que reparten puntos (`otorga_puntos = 1`).

```
select g.ronda, g.nombre as gran_premio, ts.nombre as sesion, e.apellido as piloto,
       r.estado, r.posicion, isnull(pu.puntos, 0) as puntos
from Resultado r
inner join Sesion s on s.id_sesion = r.id_sesion
inner join TipoSesion ts on ts.id_tipo_sesion = s.id_tipo_sesion
inner join GranPremio g on g.id_gp = s.id_gp
inner join Inscripcion i on i.id_inscripcion = r.id_inscripcion
inner join Empleado e on e.id_empleado = i.id_piloto
left join Puntaje pu on pu.anio = g.anio
 and pu.id_tipo_sesion = s.id_tipo_sesion and pu.posicion = r.posicion
where g.anio = 2026
  and ts.otorga_puntos = 1
order by g.ronda, ts.nombre, r.posicion
```

Con los datos de prueba: 18 filas. El abandono de Hadjar en Australia y la descalificación de Verstappen en Miami aparecen con 0 puntos.

Consulta 2. ¿Qué pilotos sumaron más de 50 puntos en 2026?

Agrupa por piloto y filtra con `having`, porque la condición es sobre la suma y no sobre cada fila. Acá Puntaje va con `inner join` a propósito: lo que no da puntos no tiene nada que sumar.

```
select e.apellido, sum(p.puntos) as puntos
from Resultado r
inner join Sesion s on s.id_sesion = r.id_sesion
inner join GranPremio g on g.id_gp = s.id_gp
inner join Inscripcion i on i.id_inscripcion = r.id_inscripcion
inner join Empleado e on e.id_empleado = i.id_piloto
inner join Puntaje p on p.anio = g.anio
 and p.id_tipo_sesion = s.id_tipo_sesion and p.posicion = r.posicion
where g.anio = 2026
group by e.apellido
having sum(p.puntos) > 50
order by puntos desc
```

Con los datos de prueba: Verstappen, con 111. Hadjar tiene 35 y queda afuera.

Consulta 3. ¿Qué circuitos tenemos cargados que no están en el calendario 2026?

Con `right join` vienen todos los circuitos, tengan o no Gran Premio. La condición del año va en el `on` y no en el `where`: si estuviera en el `where`, las filas sin Gran Premio (que tienen `g.anio` en null) se descartarían y el `right join` no serviría de nada. Después el `where g.id_gp is null` deja solo los que no tienen.

```
select c.nombre as circuito, c.ciudad
from GranPremio g
right join Circuito c on c.id_circuito = g.id_circuito and g.anio = 2026
where g.id_gp is null
```

Con los datos de prueba devuelve el Red Bull Ring, que está cargado pero no tiene Gran Premio en 2026.

Personal

Consulta 4. ¿Cuánta gente hay en cada departamento y cuánto se paga de sueldos?

Cuenta solo los contratos vigentes hoy y usa `count`, `sum` y `avg`. El promedio de un decimal sale con un montón de decimales, por eso el `cast`. Es una consulta para Recursos Humanos: por los permisos, nadie más puede leer Contrato.

```
select pu.departamento, count(*) as empleados, sum(c.sueldo_mensual_usd) as
sueldos_mensuales_usd,
       cast(avg(c.sueldo_mensual_usd) as decimal(14,2)) as sueldo_promedio_usd
from Empleado e
inner join Puesto pu on pu.id_puesto = e.id_puesto
inner join Contrato c on c.id_empleado = e.id_empleado
where c.fecha_inicio <= getdate()
  and (c.fecha_fin is null or c.fecha_fin >= getdate())
group by pu.departamento
order by sueldos_mensuales_usd desc
```

Con los datos de prueba devuelve cuatro departamentos. Deportivo es el que más cuesta, por los sueldos de los pilotos.

Consulta 5. ¿Qué contratos se vencen en los próximos 6 meses?

Para que Recursos Humanos sepa a quién tiene que renovar. Compara `fecha_fin` con la fecha de hoy, así que el resultado depende del día en que se corra.

```
select e.apellido, e.nombre, pu.nombre as puesto, c.fecha_fin,
       datediff(day, getdate(), c.fecha_fin) as dias_que_faltan
from Contrato c
inner join Empleado e on e.id_empleado = c.id_empleado
inner join Puesto pu on pu.id_puesto = e.id_puesto
where c.fecha_fin between cast(getdate() as date) and dateadd(month, 6, getdate())
order by c.fecha_fin
```

Con los datos de prueba: Tsunoda y Romero (corrida el 11/11/2026). Los dos vencen el 31/12/2026.

Consulta 6. ¿Cuántos años lleva cada empleado en la escudería?

Toma el primer contrato de cada uno con `min`, porque hay empleados con más de un contrato (Tsunoda, Walker). Los años salen de `datediff(year, ...)`, igual que la edad en el ejemplo de funciones de clase. Cuenta cambios de año calendario, así que es aproximado: alguien que entró en julio de 2025 ya figura con 1.

```
select e.apellido, e.nombre, min(c.fecha_inicio) as primer_contrato,
       datediff(year, min(c.fecha_inicio), getdate()) as anios
from Empleado e
inner join Contrato c on c.id_empleado = e.id_empleado
group by e.id_empleado, e.apellido, e.nombre
order by primer_contrato
```

Con los datos de prueba devuelve los 17 empleados, del más antiguo (Harris, desde 2021) al más nuevo (Hadjar, que entró en 2026).

Técnica

Consulta 7. ¿Qué penalizaciones de grilla hubo en 2026 y por qué?

Junta seis tablas para mostrar el Gran Premio, el piloto y la pieza de cada penalización.

```
select g.ronda, g.nombre as gran_premio, e.apellido as piloto, t.nombre as tipo_pieza,
       c.numero_serie, p.puestos_grilla, p.motivo
from Penalizacion p
inner join Inscripcion i on i.id_inscripcion = p.id_inscripcion
inner join GranPremio g on g.id_gp = i.id_gp
inner join Empleado e on e.id_empleado = i.id_piloto
inner join Componente c on c.id_componente = p.id_componente
inner join TipoComponente t on t.id_tipo = c.id_tipo
where g.anio = 2026
order by g.ronda
```

Con los datos de prueba devuelve la de Verstappen en Mónaco: cuarto MGU-K de la temporada con límite de 3, 10 puestos.

Consulta 8. ¿Qué pilotos están a una pieza del límite de algún tipo, o ya lo pasaron?

Le sirve al ingeniero para saber cuándo cambiar una pieza le va a costar puestos al piloto. Cuenta con `count(distinct ...)` porque una misma pieza se usa en varios Grandes Premios y tiene una fila de uso por cada uno. El límite sale de LimiteComponente y entra en el `group by` para poder compararlo en el `having`.

```
select e.apellido as piloto, t.nombre as tipo_pieza, l.cantidad_max as limite,
       count(distinct u.id_componente) as piezas_usadas, sum(u.km_recorridos) as km
from UsoComponente u
inner join Componente c on c.id_componente = u.id_componente
inner join TipoComponente t on t.id_tipo = c.id_tipo
inner join Inscripcion i on i.id_inscripcion = u.id_inscripcion
inner join GranPremio g on g.id_gp = i.id_gp
inner join Empleado e on e.id_empleado = i.id_piloto
inner join LimiteComponente l on l.anio = g.anio and l.id_tipo = c.id_tipo
where g.anio = 2026
group by e.apellido, t.nombre, l.cantidad_max
having count(distinct u.id_componente) >= l.cantidad_max - 1
order by piezas_usadas desc
```

Con los datos de prueba: Verstappen (4 MGU-K con límite de 3, ya penalizado) y Hadjar (2 de 3, así que el próximo MGU-K es el último sin penalización).

Consulta 9. ¿Cuántas piezas hay de cada tipo y en qué estado?

Es el stock de piezas. Agrupa por tipo y estado y suma los km de cada grupo.

```
select t.nombre as tipo, c.estado, count(*) as piezas,
       sum(c.km_acumulados) as km_totales
from Componente c
inner join TipoComponente t on t.id_tipo = c.id_tipo
group by t.nombre, c.estado
order by t.nombre, c.estado
```

Con los datos de prueba: 9 grupos. Por ejemplo, hay 4 MGU-K nuevos para usar y 4 dañados.

Finanzas

Consulta 10. ¿En qué se gastó la plata en 2026, y qué parte cuenta para el tope?

Agrupa los gastos del año por categoría y muestra si cada categoría computa para el tope.

```
select cg.nombre as categoria,
       case when cg.computa_tope = 1 then 'si' else 'no' end as cuenta_tope,
       count(*) as gastos, sum(g.monto_usd) as total_usd
from Gasto g
inner join CategoriaGasto cg on cg.id_categoria = g.id_categoria
where year(g.fecha) = 2026
group by cg.nombre, cg.computa_tope
order by total_usd desc
```

Con los datos de prueba devuelve seis categorías. La más grande es Investigación y desarrollo (69,5 millones). Hospitalidad y marketing (6,3 millones) no cuenta para el tope.

Consulta 11. ¿A qué proveedores les compramos más de 10 millones en 2026?

Agrupa por proveedor y filtra la suma con `having`.

```
select pr.razon_social, pa.nombre as pais, count(*) as compras, sum(g.monto_usd) as total_usd
from Gasto g
inner join Proveedor pr on pr.id_proveedor = g.id_proveedor
inner join Pais pa on pa.id_pais = pr.id_pais
where year(g.fecha) = 2026
group by pr.razon_social, pa.nombre
having sum(g.monto_usd) > 10000000
order by total_usd desc
```

Con los datos de prueba devuelve cuatro proveedores. El más grande es Red Bull Advanced Technologies, con 90,55 millones.

Consulta 12. ¿Qué alertas de tope saltaron y qué gasto las disparó?

TopePresupuestario se une por el año de la fecha del gasto (`year(g.fecha)`) y no por una clave foránea, porque Gasto no guarda el año.

```
select year(g.fecha) as anio, a.fecha as fecha_alerta, g.descripcion, pr.razon_social,
       g.monto_usd, a.gasto_acumulado_usd, t.monto_usd as tope_usd
from AlertaTope a
inner join Gasto g on g.id_gasto = a.id_gasto
inner join Proveedor pr on pr.id_proveedor = g.id_proveedor
inner join TopePresupuestario t on t.anio = year(g.fecha)
order by anio
```

Con los datos de prueba devuelve la de 2025: el gasto de frenos de fin de temporada llevó el acumulado a 123,7 millones sobre un tope de 135. En 2026 todavía no saltó ninguna, el año está en 89,47 % del tope, justo abajo del 90 %. El script de pruebas la hace saltar.

Subconsultas

Para cada una ponemos las tres cosas que pide el enunciado: qué información se busca, qué hace la subconsulta y por qué es adecuada.

Deportiva

Subconsulta 1. ¿Cuántos puntos saca cada piloto en promedio por fin de semana?

```
select x.piloto, count(*) as grandes_premios, sum(x.puntos) as puntos,
       cast(avg(x.puntos * 1.0) as decimal(5,2)) as promedio
from (
    select e.apellido as piloto, g.id_gp, isnull(sum(pu.puntos), 0) as puntos
    from Resultado r
    inner join Sesion s on s.id_sesion = r.id_sesion
    inner join GranPremio g on g.id_gp = s.id_gp
    inner join Inscripcion i on i.id_inscripcion = r.id_inscripcion
    inner join Empleado e on e.id_empleado = i.id_piloto
    left join Puntaje pu on pu.anio = g.anio
 and pu.id_tipo_sesion = s.id_tipo_sesion and pu.posicion = r.posicion
    where g.anio = 2026
    group by e.apellido, g.id_gp
) x
group by x.piloto
order by promedio desc
```

- Qué queremos saber: el rendimiento promedio de cada piloto por Gran Premio en 2026, con carrera y sprint juntos.

- Qué hace la subconsulta: está en el `from`. Arma una fila por piloto y Gran Premio con los puntos de ese fin de semana, y la consulta de afuera promedia esas filas.
- Por qué sirve: hay que agregar dos veces: primero sumar por Gran Premio y después promediar. Eso no se puede hacer en un solo nivel (`avg(sum(...))` da error), así que la primera agregación va adentro. El `* 1.0` es para que `avg` no haga división entera.

Con los datos de prueba: Verstappen promedia 18,50 puntos por fin de semana y Hadjar 5,83.

Subconsulta 2. ¿Cómo está hoy cada piloto: con qué contrato, cuántos Grandes Premios corrió y cuántos puntos tiene?

```
select e.apellido, p.numero_auto,
       (select c.condicion from Contrato c
        where c.id_contrato = dbo.fn_contrato_vigente(p.id_empleado, cast(getdate() as date)))
as condicion_hoy,
       (select count(*) from Inscripcion i
        inner join GranPremio g on g.id_gp = i.id_gp
        where i.id_piloto = p.id_empleado and g.anio = 2026) as gp_corridos_2026,
       dbo.fn_puntos_piloto(p.id_empleado, 2026) as puntos_2026
from Piloto p
inner join Empleado e on e.id_empleado = p.id_empleado
order by puntos_2026 desc
```

- Qué queremos saber: una ficha de cada piloto.
- Qué hace la subconsulta: hay dos en el `select` y cada una devuelve un solo valor por piloto. La primera trae la condición (titular o reserva) del contrato que devuelve `fn_contrato_vigente`. La segunda cuenta sus inscripciones de 2026. Los puntos salen de `fn_puntos_piloto`.
- Por qué sirve: cada dato viene de una tabla distinta. Si se juntara todo con JOIN, las filas se multiplicarían y el conteo daría cualquier cosa. Con una subconsulta por columna, cada número se calcula por separado.

Con los datos de prueba devuelve los cuatro pilotos. Ferraro aparece con condición NULL porque hoy no tiene contrato.

Personal

Subconsulta 3. ¿Qué empleados que no son pilotos cobran más que alguno de los pilotos?

```
select e.apellido, e.nombre, pu.nombre as puesto, c.sueldo_mensual_usd
from Empleado e
inner join Puesto pu on pu.id_puesto = e.id_puesto
inner join Contrato c on c.id_contrato = dbo.fn_contrato_vigente(e.id_empleado, cast(getdate()
as date))
where e.id_empleado not in (select id_empleado from Piloto)
  and c.sueldo_mensual_usd > any (
      select c2.sueldo_mensual_usd
      from Contrato c2
      inner join Piloto p2 on p2.id_empleado = c2.id_empleado
      where c2.id_contrato = dbo.fn_contrato_vigente(p2.id_empleado, cast(getdate() as date))
  )
```

- Qué queremos saber: si hay gente del equipo que cobra más que algún piloto.
- Qué hace la subconsulta: devuelve el sueldo vigente de cada piloto, y `> any` se cumple si el sueldo del empleado es mayor que al menos uno de esa lista. Además hay un `not in` para dejar afuera a los pilotos.
- Por qué sirve: es una comparación contra un conjunto de valores. `> any` equivale a ser mayor que el mínimo de la lista, pero escrito así se lee igual que la pregunta.

Con los datos de prueba: Wilson, el director de equipo (95.000 por mes), que cobra más que Tsunoda como reserva (90.000).

Subconsulta 4. ¿A quién se le cambió el sueldo en 2026?

```
select e.apellido, e.nombre, c.sueldo_mensual_usd as sueldo_actual
from Empleado e
inner join Contrato c on c.id_empleado = e.id_empleado
where exists (
    select 1
    from AuditoriaContrato a
    where a.id_contrato = c.id_contrato
      and year(a.fecha) = 2026
)
```

- Qué queremos saber: qué empleados tuvieron algún cambio de sueldo este año.
- Qué hace la subconsulta: para cada contrato busca si hay un registro de 2026 en AuditoriaContrato. Es correlacionada por `id_contrato`.
- Por qué sirve: solo importa si hubo al menos un cambio, no cuántos. Con un JOIN, un contrato con dos cambios saldría dos veces.

Con los datos de prueba: Verstappen y Romero.

Técnica

Subconsulta 5. ¿Qué piezas llevan más km que el promedio de su tipo?

```
select c.numero_serie, t.nombre as tipo, c.km_acumulados
from Componente c
inner join TipoComponente t on t.id_tipo = c.id_tipo
where c.km_acumulados > (
    select avg(c2.km_acumulados)
    from Componente c2
    where c2.id_tipo = c.id_tipo
)
order by t.nombre, c.km_acumulados desc
```

- Qué queremos saber: qué piezas se están gastando más rápido que las demás de su tipo.
- Qué hace la subconsulta: calcula el promedio de km de las piezas del mismo tipo que la pieza de la fila actual.
- Por qué sirve: el promedio cambia según el tipo, así que no se puede calcular una sola vez para toda la consulta.

Con los datos de prueba: 9 piezas. Las que más preocupan son las cajas GBX-26-01 y GBX-26-02, que están cerca de su vida útil.

Subconsulta 6. ¿Cuál es la pieza más gastada de cada tipo?

```
select t.nombre as tipo, c.numero_serie, c.km_acumulados, c.estado
from Componente c
inner join TipoComponente t on t.id_tipo = c.id_tipo
where c.km_acumulados >= all (
    select c2.km_acumulados
    from Componente c2
    where c2.id_tipo = c.id_tipo
)
order by t.nombre
```

- Qué queremos saber: la pieza con más km de cada tipo, con su número de serie y su estado.
- Qué hace la subconsulta: trae los km de todas las piezas del mismo tipo, y `>= all` deja la que tiene tantos km o más que todas.

- Por qué sirve: con `group by` y `max` se saca el máximo, pero no a qué pieza corresponde. Con `>= all` sale la fila entera. Si hubiera empate saldrían las dos.

Con los datos de prueba devuelve una por tipo. La caja GBX-26-01 tiene 3200 de sus 3500 km, y es la que se retira cuando se cierra Mónaco.

Finanzas

Subconsulta 7. ¿A qué proveedores no les compramos nada en 2026?

```
select pr.razon_social, pr.identificacion_fiscal
from Proveedor pr
where pr.id_proveedor not in (
    select g.id_proveedor
    from Gasto g
    where year(g.fecha) = 2026
)
```

- Qué queremos saber: los proveedores sin compras en el año, por ejemplo para revisar si conviene seguir con ellos.
- Qué hace la subconsulta: devuelve los proveedores que tienen algún gasto en 2026, y `not in` deja los demás.
- Por qué sirve: es una lista sin correlación. Como `id_proveedor` es NOT NULL en Gasto no aparece el problema de `not in` con los nulos (si la lista tuviera un NULL, la consulta no devolvería nada).

Con los datos de prueba: Mobil 1.

Subconsulta 8. ¿Qué gastos se llevaron, solos, más del 10 % del tope de su año?

```
select year(g.fecha) as anio, g.fecha, g.descripcion, g.monto_usd
from Gasto g
where g.monto_usd > (
    select t.monto_usd * 0.10
    from TopePresupuestario t
    where t.anio = year(g.fecha)
)
order by g.monto_usd desc
```

- Qué queremos saber: los gastos más grandes, los que más mueven el tope.
- Qué hace la subconsulta: para cada gasto busca el tope de su año y calcula el 10 %. Es correlacionada por el año del gasto.
- Por qué sirve: el tope es distinto cada año (135 millones en 2025 y 215 en 2026), así que el límite cambia de fila a fila y no se puede poner un número fijo.

Con los datos de prueba: 7 gastos, 4 de 2025 y 3 de 2026. El más grande es el desarrollo aerodinámico del RB21, de 45 millones.

7. Vistas, procedimientos, triggers, cursor y funciones

Están en los scripts 03 a 06. Las secciones van en el orden del enunciado, aunque en los scripts las funciones se crean primero porque las vistas y los procedimientos las usan.

De cada objeto ponemos lo que pide el enunciado para su tipo y qué regla de negocio del alcance hace cumplir (RN01 a RN22).

Vistas

Están en `scripts/04_vistas.sql`. Hay una por área.

vw_puntos_equipo_por_gp

Objetivo	Seguir cómo le va al equipo en la temporada. Como no guardamos a los rivales no hay tabla del campeonato, pero sí se puede ver cuántos puntos sumó el equipo en cada Gran Premio y cuántos lleva en total.
Información que muestra	Año, ronda y Gran Premio, los puntos del equipo en ese fin de semana (los dos pilotos, carrera y sprint) y el acumulado de la temporada hasta esa ronda.
Tablas	GranPremio, Sesion, Resultado y Puntaje
Por qué es una vista	Es una consulta que se usa siempre igual y tiene dos niveles de cálculo (sumar por Gran Premio y después ir acumulando), que no queremos repetir cada vez. También deja que el rol de consulta vea los puntos sin tener permiso sobre Resultado ni Puntaje.

```
create or alter view vw_puntos_equipo_por_gp
as
select x.anio, x.ronda, x.gran_premio, x.puntos,
       sum(x.puntos) over (partition by x.anio order by x.ronda) as acumulado
from (
    select g.anio, g.ronda, g.nombre as gran_premio, isnull(sum(p.puntos), 0) as puntos
    from GranPremio g
    inner join Sesion s on s.id_gp = g.id_gp
    inner join Resultado r on r.id_sesion = s.id_sesion
    left join Puntaje p on p.anio = g.anio and p.id_tipo_sesion = s.id_tipo_sesion and
p.posicion = r.posicion
    group by g.anio, g.ronda, g.nombre
) x
```

La subconsulta suma los puntos de cada Gran Premio. Puntaje va con `left join` para que un fin de semana sin puntos aparezca igual, con 0. Afuera, `sum() over` es la función de ventana que vimos en clase: va sumando ronda por ronda y arranca de cero en cada temporada (`partition by anio`). Un Gran Premio aparece en la vista cuando ya tiene resultados cargados.

Con los datos de prueba, `select * from vw_puntos_equipo_por_gp` devuelve las 6 fechas corridas y el equipo llega a Mónaco con 146 puntos.

vw_nomina

Objetivo	Mostrar quién trabaja hoy en la escudería sin mostrar los sueldos (RN21).
Información que muestra	Empleado, puesto, departamento, nacionalidad, desde y hasta cuándo va su contrato y, si es piloto, si es titular o reserva.
Tablas	Empleado, Puesto, Pais y Contrato, más la función `fn_contrato_vigente`
Por qué es una vista	Es la forma de que los demás roles vean el plantel sin poder leer Contrato, que tiene los sueldos. Los roles tienen permiso sobre la vista y DENY sobre Contrato. La vista igual puede leer Contrato porque las dos tienen el mismo dueño (`dbo`): SQL Server no vuelve a revisar los permisos sobre la tabla (encadenamiento de propiedad).

```
create or alter view vw_nomina
as
select e.id_empleado, e.apellido, e.nombre,
       p.nombre as puesto, p.departamento, pa.nombre as nacionalidad,
       c.fecha_inicio as contrato_desde, c.fecha_fin as contrato_hasta, c.condicion
from Empleado e
inner join Puesto p on p.id_puesto = e.id_puesto
inner join Pais pa on pa.id_pais = e.id_pais
inner join Contrato c on c.id_contrato = dbo.fn_contrato_vigente(e.id_empleado, cast(getdate()
as date))
```

El join contra la función deja solo a los que tienen contrato vigente hoy. Con los datos de prueba salen 15 empleados. Ferraro y Walker no aparecen porque hoy no tienen contrato.

vw_estado_piezas

Objetivo	Ver de un vistazo cuánto le queda a cada pieza, para decidir qué se pone en el auto antes de cada Gran Premio.
Información que muestra	Pieza, tipo, estado, km acumulados, vida útil, km restantes y porcentaje usado.
Tablas	Componente y TipoComponente, más la función `fn_km_restantes`
Por qué es una vista	Junta un cálculo (los km restantes) y un porcentaje que el ingeniero necesita siempre juntos. El ingeniero de pista tiene permiso de lectura sobre la vista.

```
create or alter view vw_estado_piezas
as
select c.id_componente, c.numero_serie, t.nombre as tipo, c.estado,
       c.km_acumulados, t.vida_util_km,
       dbo.fn_km_restantes(c.id_componente) as km_restantes,
       cast(c.km_acumulados * 100 / t.vida_util_km as decimal(5,1)) as pct_usado
from Componente c
inner join TipoComponente t on t.id_tipo = c.id_tipo
```

Con `select * from vw_estado_piezas order by km_restantes` la primera es la caja GBX-26-01, a la que le quedan 300 km.

vw_situacion_tope

Objetivo	Saber en todo momento cuánto se gastó en el año y cuánto queda antes del tope.
Información que muestra	Año, tope, gasto computable, disponible, porcentaje usado y umbral de aviso.
Tablas	TopePresupuestario, y Gasto y CategoriaGasto a través de la función `fn_gasto_computable`
Por qué es una vista	El cálculo de qué gastos cuentan para el tope vive en la función, y la vista lo presenta para cada año. Finanzas tiene permiso sobre la vista y el rol de consulta no, porque son montos.

```
create or alter view vw_situacion_tope
as
select t.anio, t.monto_usd as tope_usd,
       dbo.fn_gasto_computable(t.anio) as gastado_usd,
       t.monto_usd - dbo.fn_gasto_computable(t.anio) as disponible_usd,
       cast(dbo.fn_gasto_computable(t.anio) * 100 / t.monto_usd as decimal(5,2)) as pct_usado,
       t.umbral_alerta_pct
from TopePresupuestario t
```

Con los datos de prueba, 2025 terminó en 91,63 % del tope (con una alerta) y 2026 va en 89,47 %, todavía sin alerta.

Procedimientos

Están en `scripts/05_procedimientos.sql`. Todos siguen la misma estructura, la que usamos en clase:

```
create or alter procedure usp_ejemplo
    @param int
as
begin
    set nocount on
    set xact_abort on

    begin try
        if not exists (select 1 from Tabla where id = @param)
            raiserror('el registro no existe', 16, 1)

        begin transaction

        insert into ...
        update ...

        commit transaction

        print 'listo'
    end try
    begin catch
        if xact_state() <> 0
            rollback transaction

        print 'error: ' + error_message()
    end catch
end
```

Primero se valida lo que ninguna restricción controla. Cada validación que falla corta con `raiserror` y salta al `catch`. Lo que ya controla una FK, un CHECK o un UNIQUE no se vuelve a validar: si falla, el `catch` muestra el mensaje de SQL Server, que nombra la restricción.

La transacción la abren solo los que modifican más de una cosa, `usp_cerrar_gp` y `usp_registrar_compra`. Si algo falla adentro (una restricción, un trigger que rechaza), el `catch` deshace lo que haya quedado abierto, y `set xact_abort on` hace que cualquier error marque la transacción para deshacerse entera. Los que hacen un solo INSERT no la necesitan, porque si el INSERT falla no queda nada hecho.

usp_cargar_resultado

Parámetros	`@idsesion int`, `@idinsc int`, `@pos tinyint = null`, `@estado nvarchar(15) = 'Finalizo'`
Qué hace	Carga el resultado de un piloto inscripto en una sesión.
Validaciones	Que la sesión y la inscripción sean del mismo Gran Premio (RN10) y que la posición no la tenga otro piloto (RN11). Que existan lo controlan las FK, que el piloto no tenga ya un resultado en la sesión la clave primaria, y que la posición sea coherente con el estado el CHECK de Resultado (RN11).
Qué modifica	Inserta una fila en Resultado.
Errores	Las dos validaciones tienen su mensaje. Si salta una restricción, el `catch` muestra el mensaje de SQL Server.

RN10 no se puede escribir como restricción sin agregarle `id_gp` a Resultado, y eso sería una dependencia transitiva (rompe la 3FN). Por eso se controla acá, y los permisos hacen que este procedimiento sea la única forma de cargar un resultado (RN22).

usp_inscribir_piloto

Parámetros	`@idgp int`, `@idpiloto int`, `@idchasis int`
Qué hace	Inscribe a un piloto en un Gran Premio con un chasis.
Validaciones	Que el Gran Premio exista, que el piloto tenga contrato vigente en la fecha del Gran Premio (RN04, con `fn_contrato_vigente`), que el Gran Premio tenga menos de dos inscriptos (RN03) y que el chasis exista y sea de la misma temporada (RN06). Que el piloto exista lo controla la FK, y el mismo piloto dos veces o el mismo chasis para dos pilotos, los UNIQUE de Inscripcion (RN05).
Qué modifica	Inserta una fila en Inscripcion.
Errores	Igual que el anterior: mensaje propio por cada validación, y el de SQL Server si salta una restricción.

usp_retirar_componente

Parámetros	`@idcomp int`, `@danada bit = 0`
Qué hace	Saca una pieza de servicio. La deja en 'Retirado' si llegó a su vida útil o en 'Danado' si se rompió.
Validaciones	Que la pieza exista y que no esté ya retirada.
Qué modifica	El estado de la pieza en Componente.
Errores	Es el más simple. Hace un solo UPDATE, así que no necesita transacción ni `try`. Si la pieza no existe o ya está retirada, avisa con `print` y sale con `return`.

Lo llama el cierre de Gran Premio para cada pieza que llega a su vida útil.

usp_cerrar_gp

Parámetros	`@idgp int`
Qué hace	Cierra un Gran Premio: a cada pieza usada le suma los km, pasa las nuevas a 'En uso', retira las que llegaron a su vida útil y marca el Gran Premio como cerrado (RN14). Es el procedimiento del cursor, que se explica más abajo.
Validaciones	Que el Gran Premio exista y que no estuviera cerrado (RN15), las dos en una sola validación.
Qué modifica	`km_acumulados` y `estado` en Componente, y `cerrado` en GranPremio.
Errores	Todo pasa en una sola transacción. Si algo falla a la mitad, no queda ningún km sumado ni el Gran Premio cerrado.

Es el único que modifica `km_acumulados`, que es la desnormalización explicada en la normalización. Ningún rol tiene permiso de UPDATE sobre Componente.

usp_registrar_compra

Parámetros	`@idtipo int, @serie nvarchar(30), @fecha date, @monto decimal(14,2), @descr nvarchar(200), @idcat int, @idprov int`
Qué hace	Da de alta una pieza nueva y el gasto de su compra, juntos.
Validaciones	Ninguna propia. El monto lo controla el CHECK de Gasto; el tipo, la categoría y el proveedor, las FK; el número de serie repetido, el UNIQUE de Componente, y el tope (RN19), el trigger de Gasto.
Qué modifica	Inserta en Componente y en Gasto, y si el gasto cruza el umbral el trigger inserta en AlertaTope.
Errores	Es donde más importa la transacción: si el trigger rechaza el gasto por el tope, la pieza tampoco queda creada. Lo prueba el script de pruebas con la caja GBX-26-04.

El id de la pieza nueva va con `@@identity` en el insert de Gasto, que es lo que se ejecuta justo después de insertar en Componente. Leerlo más tarde no serviría: el trigger de Gasto puede insertar una alerta, y `@@identity` pasaría a tener el id de la alerta.

usp_registrar_gasto

Parámetros	`@fecha date, @monto decimal(14,2), @descr nvarchar(200), @idcat int, @idprov int`
Qué hace	Registra un gasto que no es la compra de una pieza (fletes, viajes, hospitalidad).
Validaciones	Ninguna propia. El monto lo controla el CHECK de Gasto, la categoría y el proveedor las FK, y el tope el trigger.
Qué modifica	Inserta en Gasto, y el trigger puede insertar en AlertaTope.
Errores	Hace un solo INSERT, así que no abre transacción. Si una restricción o el trigger lo rechazan (año sin tope o se pasa del tope), el `catch` muestra el mensaje.

Triggers

Están en `scripts/06_triggers.sql`. Los cuatro son AFTER y trabajan con toda la tabla `inserted`, no con una sola fila: si un INSERT trae varias filas y una rompe la regla, se rechaza el INSERT completo.

Cuando rechazan algo hacen `rollback transaction`, después `raiserror` y `return`. Así se deshace la operación que los disparó y el mensaje que llega es el nuestro.

Usamos triggers para las reglas que tienen que valer siempre, venga de donde venga el cambio. Las tablas donde los roles escriben directo (Sesion, Contrato, UsoComponente) quedan protegidas igual que si usaran un procedimiento. Y en Gasto, aunque solo se escribe por procedimiento, el control del tope está en el trigger para que valga también para el administrador.

tr_Sesion_validar

Evento	AFTER INSERT, UPDATE
Tabla	Sesion
Condición	Alguna sesión nueva o modificada es de un tipo sprint en un Gran Premio sin formato sprint, o su fecha cae fuera del fin de semana del Gran Premio.
Acción	Deshace la operación y avisa cuál de las dos reglas se rompió.
Regla	RN01 y RN02

Un CHECK no sirve porque las dos reglas miran datos de otras tablas (TipoSesion y GranPremio).

tr_Contrato

Evento	AFTER INSERT, UPDATE
Tabla	Contrato
Condición	El contrato se superpone con otro del mismo empleado, o dice titular/reserva y el empleado no es piloto, o es de un piloto y no lo dice.
Acción	Si se rompe alguna regla, deshace y avisa. Si no, por cada contrato al que le cambió el sueldo inserta una fila en AuditoriaContrato con el sueldo anterior (de `deleted`) y el nuevo (de `inserted`). La fecha y el usuario los pone el default de la tabla.
Regla	RN07, RN08 y RN09

Para la superposición, un contrato sin fecha de fin se compara como si terminara el 31/12/9999. Recursos Humanos modifica Contrato directo, y el trigger asegura que ningún cambio de sueldo quede sin registrar.

tr_UsoComponente

Evento	AFTER INSERT
Tabla	UsoComponente
Condición	Se usa una pieza retirada o dañada, o el Gran Premio ya está cerrado. Y para las penalizaciones: una pieza que se usa por primera vez hace que el piloto pase el límite de su tipo en la temporada.
Acción	En los dos primeros casos deshace y avisa. Si no, inserta en Penalizacion una fila por cada pieza de más: 10 puestos la primera y 5 cada una de las siguientes.
Regla	RN13, RN15 y RN16

Las penalizaciones son lo más difícil de los cuatro triggers, porque un mismo INSERT puede traer varias piezas. Para cada fila de `inserted` el trigger mira si la pieza es nueva, o sea si ese es su único uso. Si lo es, cuenta con `count(distinct ...)` cuántas piezas de ese tipo usó el piloto en la temporada: las que ya venía usando, más las nuevas de este INSERT con un id menor o igual. Si ese número pasa el límite de LimiteComponente, inserta la penalización: 10 puestos si es justo el límite más uno y 5 si es más.

Así, si en un mismo INSERT entran dos MGU-K que pasan el límite, el primero recibe 10 puestos y el segundo 5. El script de pruebas lo hace con Hadjar.

tr_Gasto_tope

Evento	AFTER INSERT, UPDATE
Tabla	Gasto
Condición	El año del gasto no tiene tope cargado, la pieza del gasto ya tenía un gasto de compra, o con el gasto el total computable del año pasa el tope. Y para la alerta: el total del año llegó al porcentaje de aviso y ese año todavía no tiene alerta.
Acción	En los tres primeros casos deshace y avisa. Si no, y corresponde, inserta en AlertaTope el gasto y el acumulado del año.
Regla	RN17, RN19 y RN20, con el cálculo de RN18

La alerta sale una sola vez por año, cuando el total computable (`fn_gasto_computable`) llega al umbral. Si un mismo INSERT trae varios gastos, queda en el de id más alto.

RN17 está acá y no en un UNIQUE porque Gasto tiene muchos gastos sin pieza (`id_componente` en NULL), y un UNIQUE de SQL Server acepta un solo NULL.

Cursor

Está dentro de `usp_cerrar_gp`.

Problema	Al cerrar un Gran Premio hay que pasar por cada pieza que se usó, sumarle los km que hizo y, si llegó a su vida útil, retirarla.
Por qué un cursor	Para cada pieza que llega a su vida útil hay que ejecutar otro procedimiento, `usp_retirar_componente`. Con un UPDATE se podrían sumar los km de todas las piezas de una vez, pero no se puede hacer un EXEC por cada fila. Además el retiro queda en un solo lugar: si mañana cambia cómo se retira una pieza, el cierre no hay que tocarlo.
Registros que recorre	Los usos (UsoComponente) de las inscripciones de ese Gran Premio, con la pieza y los km. Son dos pilotos con unas cuatro piezas cada uno, así que ir fila por fila no cuesta nada.
Qué hace con cada uno	Le suma los km a la pieza y la pasa de 'Nuevo' a 'En uso'. Si con eso llegó a la vida útil de su tipo, ejecuta el retiro.

```
declare cur_usos cursor local fast_forward for
    select u.id_componente, u.km_recorridos
    from UsoComponente u
    inner join Inscripcion i on i.id_inscripcion = u.id_inscripcion
    where i.id_gp = @idgp

open cur_usos
fetch next from cur_usos into @idcomp, @km

while @@fetch_status = 0
begin
    update Componente
    set km_acumulados = km_acumulados + @km,
        estado = case when estado = 'Nuevo' then 'En uso' else estado end
    where id_componente = @idcomp

    if exists (select 1 from Componente c inner join TipoComponente t on t.id_tipo = c.id_tipo
               where c.id_componente = @idcomp and c.km_acumulados >= t.vida_util_km and
c.estado <> 'Retirado')
        exec usp_retirar_componente @idcomp = @idcomp

    fetch next from cur_usos into @idcomp, @km
end

close cur_usos
deallocate cur_usos
```

Está declarado `local`, así solo existe dentro del procedimiento, y `fast_forward`, que es de solo lectura y solo para adelante: es lo único que necesita y es el más liviano.

Con los datos de prueba, cerrar Mónaco recorre 8 usos. La caja GBX-26-01 tenía 3200 km, hizo 540 más y pasó los 3500 de vida útil, así que el cursor la retira.

Funciones

Están en `scripts/03_funciones.sql`. Cuatro son escalares y una es de tabla.

fn_puntos_piloto

Objetivo	Los puntos de un piloto en una temporada. Como los puntos no se guardan (RN12), hay que calcularlos.
Parámetros	`@idpiloto int`, `@anio smallint`
Retorno	`int`
Lógica	Suma los puntos de Puntaje cruzando cada resultado del piloto por temporada, tipo de sesión y posición. Si no sumó nada devuelve 0 en vez de NULL.
Ejemplo	`select dbo.fn_puntos_piloto(1, 2026)` devuelve 111, los puntos de Verstappen.

```
create or alter function dbo.fn_puntos_piloto (@idpiloto int, @anio smallint)
returns int
as
begin
    return (
        select isnull(sum(p.puntos), 0)
        from Resultado r
        inner join Sesion s on s.id_sesion = r.id_sesion
        inner join GranPremio g on g.id_gp = s.id_gp
        inner join Inscripcion i on i.id_inscripcion = r.id_inscripcion
        inner join Puntaje p on p.anio = g.anio and p.id_tipo_sesion = s.id_tipo_sesion and
p.posicion = r.posicion
        where i.id_piloto = @idpiloto and g.anio = @anio
    )
end
```

fn_contrato_vigente

Objetivo	Saber qué contrato tenía vigente un empleado en una fecha.
Parámetros	`@idemp int`, `@fecha date`
Retorno	`int`: el id del contrato, o NULL si en esa fecha no tenía
Lógica	Busca el contrato que empezó antes o en esa fecha y que no tiene fin o termina después. Como RN07 no deja contratos superpuestos, hay uno o ninguno.
Ejemplo	`select dbo.fn_contrato_vigente(3, '2025-06-01')` devuelve 3 y con `'2026-06-01'` devuelve 4: Tsunoda pasó de titular a reserva. `select dbo.fn_contrato_vigente(4, '2026-06-01')` devuelve NULL, Ferraro ya no tiene contrato.

```
create or alter function dbo.fn_contrato_vigente (@idemp int, @fecha date)
returns int
as
begin
    return (
        select id_contrato
        from Contrato
        where id_empleado = @idemp and fecha_inicio <= @fecha and (fecha_fin is null or
fecha_fin >= @fecha)
    )
end
```

La usan `usp_inscribir_piloto` (RN04), `vw_nomina` y dos de las subconsultas.

fn_km_restantes

Objetivo	Cuántos km le quedan a una pieza antes de llegar a su vida útil.
Parámetros	`@idcomp int`
Retorno	`decimal(10,2)`
Lógica	Vida útil del tipo menos km acumulados. Con `iif`, si la pieza ya pasó su vida útil devuelve 0 en vez de un número negativo.
Ejemplo	`select dbo.fn_km_restantes(9)` devuelve 300 (la caja GBX-26-01).

```
create or alter function dbo.fn_km_restantes (@idcomp int)
returns decimal(10,2)
as
begin
    return (
        select iif(c.km_acumulados > t.vida_util_km, 0, t.vida_util_km - c.km_acumulados)
        from Componente c
        inner join TipoComponente t on t.id_tipo = c.id_tipo
        where c.id_componente = @idcomp
    )
end
```

fn_gasto_computable

Objetivo	El gasto de un año que cuenta para el tope (RN18).
Parámetros	`@anio smallint`
Retorno	`decimal(14,2)`
Lógica	Suma los gastos del año que son de categorías con `computa_tope = 1`.
Ejemplo	`select dbo.fn_gasto_computable(2026)` devuelve 192.350.000.

```
create or alter function dbo.fn_gasto_computable (@anio smallint)
returns decimal(14,2)
as
begin
    return (
        select isnull(sum(g.monto_usd), 0)
        from Gasto g
        inner join CategoriaGasto c on c.id_categoria = g.id_categoria
        where c.computa_tope = 1 and year(g.fecha) = @anio
    )
end
```

La usan el trigger del tope y la vista del tope. Que el cálculo esté en un solo lugar asegura que los dos digan lo mismo.

fn_piezas_piloto

Objetivo	Para un piloto y una temporada, cuántas piezas de cada tipo usó, cuál es el límite y cuántas le quedan.
Parámetros	`@idpiloto int`, `@anio smallint`
Retorno	Una tabla con `tipo`, `limite`, `usadas` y `disponibles` (función de tabla en línea)
Lógica	Arranca de los límites de ese año, así aparecen también los tipos que el piloto todavía no usó, y les une con `left join` cuántas piezas distintas de cada tipo usó. Si `disponibles` da negativo, el piloto ya se pasó.
Ejemplo	`select * from dbo.fn_piezas_piloto(1, 2026)` devuelve los cuatro tipos. En MGU-K dice límite 3, usadas 4 y disponibles -1.

```
create or alter function dbo.fn_piezas_piloto (@idpiloto int, @anio smallint)
returns table
as
return (
    select t.nombre as tipo, l.cantidad_max as limite, isnull(x.usadas, 0) as usadas,
           l.cantidad_max - isnull(x.usadas, 0) as disponibles
    from LimiteComponente l
    inner join TipoComponente t on t.id_tipo = l.id_tipo
    left join (
        select c.id_tipo, count(distinct u.id_componente) as usadas
        from UsoComponente u
        inner join Componente c on c.id_componente = u.id_componente
        inner join Inscripcion i on i.id_inscripcion = u.id_inscripcion
        inner join GranPremio g on g.id_gp = i.id_gp
        where i.id_piloto = @idpiloto and g.anio = @anio
        group by c.id_tipo
    ) x on x.id_tipo = l.id_tipo
    where l.anio = @anio
)
```

Es una función de tabla y no una vista porque necesita parámetros, y una vista no los recibe.

8. Manejo de errores, seguridad y centralización

Manejo de errores

Cómo se manejan

Hay tres lugares donde un error se frena, y cada uno lo hace a su manera.

Las restricciones de las tablas (CHECK, UNIQUE y FK) son la primera barrera. No hay que programar nada: si un dato no cumple, SQL Server rechaza la instrucción con un mensaje que nombra la restricción (por eso todas tienen nombre).

Los procedimientos siguen la estructura que vimos en clase. Validan lo que ninguna restricción controla y, si algo no cierra, cortan con `raiserror('mensaje', 16, 1)`. La severidad 16 es la de un error que el usuario puede corregir, y hace saltar al `catch`. Los que modifican más de una cosa (`usp_cerrar_gp` y `usp_registrar_compra`) trabajan dentro de `begin transaction` y `commit`, con `set xact_abort on`, y en el `catch` deshacen la transacción si quedó abierta (`if xact_state() <> 0 rollback transaction`). Todos muestran el error con `print 'error: ' + error_message()`.

Los triggers no tienen `try`. Cuando una regla no se cumple hacen `rollback transaction`, `raiserror` con nuestro mensaje y `return`. El rollback deshace la instrucción que los disparó, así que nunca queda nada a medias. Si la instrucción venía de un procedimiento o de un bloque `try`, el `catch` recibe nuestro mensaje. Si alguien la corre suelta en SSMS, ve nuestro mensaje y abajo el error 3609 de SQL Server ("The transaction ended in the trigger. The batch has been aborted."), que solo avisa que el lote se cortó. En los dos casos los datos quedan como estaban.

Los mensajes propios van en castellano y en minúscula, y dicen qué pasó: "el Gran Premio ya tiene dos pilotos inscriptos".

Las seis situaciones

Son las que pide el enunciado. Todos los ejemplos están en `scripts/09_pruebas.sql`, y el mensaje es el que se ve al correrlo.

Situación	Ejemplo	Quién lo frena	Mensaje
Datos inválidos	Un resultado 'Finalizo' sin posición	CHECK `CK_Resultado_posicion`	The INSERT statement conflicted with the CHECK constraint "CK_Resultado_posicion"
	Una compra con monto negativo	CHECK `CK_Gasto_monto`	The INSERT statement conflicted with the CHECK constraint "CK_Gasto_monto"
	Una sesión fuera del fin de semana del Gran Premio	Trigger de Sesion	la fecha de la sesion esta fuera del fin de semana del Gran Premio
Violaciones de restricciones	El mismo chasis para dos pilotos	UNIQUE `UQ_Inscripcion_gp_chasis`	Violation of UNIQUE KEY constraint 'UQ_Inscripcion_gp_chasis'
	Un número de serie repetido	UNIQUE `UQ_Componente_serie`	Violation of UNIQUE KEY constraint 'UQ_Componente_serie'
Registros inexistentes	Cerrar el Gran Premio 99	Validación de `usp_cerrar_gp`	el Gran Premio no existe o ya estaba cerrado

Situación	Ejemplo	Quién lo frena	Mensaje
	Cargar un resultado en la sesión 999	FK `FK_Resultado_Sesion`	The INSERT statement conflicted with the FOREIGN KEY constraint "FK_Resultado_Sesion"
	Un gasto de un proveedor que no existe	FK `FK_Gasto_Proveedor`	The INSERT statement conflicted with the FOREIGN KEY constraint "FK_Gasto_Proveedor"
Parámetros incorrectos	Inscribir con un chasis de 2025 en un Gran Premio de 2026	Validación de `usp_inscribir_piloto`	el chasis no existe o no es de la temporada del Gran Premio
	Cargar el resultado de una inscripción de otro Gran Premio	Validación de `usp_cargar_resultado`	la inscripcion no es del mismo Gran Premio que la sesion
	Dos pilotos en la misma posición de una sesión	Validación de `usp_cargar_resultado`	esa posicion ya la tiene otro piloto en la sesion
Operaciones no permitidas	Inscribir un tercer piloto	Validación de `usp_inscribir_piloto`	el Gran Premio ya tiene dos pilotos inscriptos
	Cerrar dos veces el mismo Gran Premio	Validación de `usp_cerrar_gp`	el Gran Premio no existe o ya estaba cerrado
	Usar una pieza retirada	Trigger de UsoComponente	no se puede usar una pieza retirada o danada
	Un contrato superpuesto con otro	Trigger de Contrato	el empleado ya tiene un contrato que se superpone con esas fechas
	Dos gastos de compra para la misma pieza	Trigger de Gasto	esa pieza ya tiene un gasto de compra
	Escribir directo en Resultado sin pasar por el procedimiento	Permisos (DENY)	The INSERT permission was denied on the object 'Resultado'
Errores durante transacciones	Comprar una pieza cuyo gasto pasa el tope	El trigger de Gasto rechaza y el `catch` del procedimiento deshace todo	el gasto hace pasar el tope presupuestario del anio

El último caso es el que muestra para qué sirve la transacción. `usp_registrar_compra` inserta la pieza y después el gasto. El gasto pasa el tope, el trigger lo rechaza y la pieza, que ya estaba insertada, también se deshace: después de la prueba, la caja GBX-26-04 no existe. Lo mismo pasa con el cierre de Gran Premio: si algo falla a la mitad, no queda ningún km sumado ni el Gran Premio cerrado.

Seguridad

Está en `scripts/07_seguridad.sql`.

Usuarios y roles

Cada actor del alcance es un rol de la base. Hay un login por rol en el servidor y un usuario en la base para cada login. Los permisos se dan a los roles, nunca a los usuarios, así que para sumar a alguien alcanza con agregarlo al rol.

Rol	Usuario y login	Quién es
`rol_director_deportivo`	`usr_director`, `login_director`	Arma el calendario, inscribe a los pilotos y carga los resultados
`rol_ingeniero_pista`	`usr_ingeniero`, `login_ingeniero`	Registra el uso de piezas y cierra los Grandes Premios
`rol_rrhh`	`usr_rrhh`, `login_rrhh`	Empleados, contratos y sueldos
`rol_finanzas`	`usr_finanzas`, `login_finanzas`	Compras, gastos y el tope
`rol_consulta`	`usr_consulta`, `login_consulta`	Solo mira información general

Los datos fijos (países, circuitos, tipos de sesión, tabla de puntos, tipos de pieza, límites y topes) los carga el administrador de la base, así que ningún rol puede modificarlos.

Las contraseñas están escritas en el script porque es un trabajo práctico y hay que poder probarlo. En una base de verdad irían con autenticación de Windows o se pedirían al crear el login.

Permisos de cada rol

Rol	Puede	No puede
Director deportivo	Leer el calendario y los catálogos deportivos. Cargar Grandes Premios y sesiones. Ejecutar `usp_cargar_resultado` y `usp_inscribir_piloto`. Ver las vistas de puntos, nómina y piezas	Modificar o borrar Grandes Premios. Escribir directo en Resultado o Inscripcion. Ver Contrato, AuditoriaContrato o Gasto
Ingeniero de pista	Leer Grandes Premios, inscripciones, chasis y piezas. Cargar usos de piezas. Ejecutar `usp_cerrar_gp` y `usp_retirar_componente`. Ver `vw_estado_piezas` y `fn_piezas_piloto`	Modificar o borrar usos. Escribir en Componente o Penalizacion. Modificar Grandes Premios. Ver Contrato o Gasto
Recursos Humanos	Cargar y modificar empleados, pilotos y contratos. Leer la auditoría de sueldos	Borrar contratos. Escribir en AuditoriaContrato. Ver Gasto
Finanzas	Leer gastos, piezas, topes y alertas. Cargar y modificar proveedores. Ejecutar `usp_registrar_compra` y `usp_registrar_gasto`. Ver `vw_situacion_tope`	Escribir directo en Gasto, AlertaTope o Componente. Ver Contrato o AuditoriaContrato
Consulta	Leer el calendario, los circuitos y las sesiones. Ver `vw_puntos_equipo_por_gp` y `vw_nomina`	Ver Contrato, AuditoriaContrato, Gasto o `vw_situacion_tope`

Lo que no está en "Puede" no se le dio, y sin GRANT SQL Server lo rechaza. DENY explícito hay solo para las dos reglas de permisos: leer Contrato, para todos los roles menos Recursos Humanos (RN21), y escribir directo en Resultado, Inscripcion, Gasto y Componente, para los roles que las modifican por procedimiento (RN22). Ahí el DENY deja la regla escrita y, como gana siempre sobre GRANT, sigue valiendo aunque mañana alguien quede en dos roles o se le dé un permiso de más.

Los dos patrones

Para escribir datos hay dos caminos, y cada tabla usa uno.

Solo por procedimiento. En Resultado, Inscripcion, Gasto y Componente el rol tiene EXECUTE sobre el procedimiento y DENY de INSERT, UPDATE y DELETE sobre la tabla (RN22). Funciona porque el procedimiento y la tabla tienen el mismo dueño (`dbo`): cuando el procedimiento escribe en la tabla, SQL Server no vuelve a revisar los permisos de quien lo ejecuta. Eso se llama encadenamiento de propiedad, y es lo que deja escribir por el procedimiento y no por fuera. Lo usamos cuando la operación tiene validaciones que una restricción no puede hacer (RN03, RN04, RN06, RN10) o varios pasos que tienen que salir juntos (la pieza y su gasto).

Escritura directa controlada por trigger. En Sesion, Contrato y UsoComponente el rol tiene permiso de INSERT o UPDATE sobre la tabla, y el trigger controla las reglas. Lo usamos cuando la operación es una carga simple pero la regla tiene que valer igual por cualquier camino.

Información sensible

- Sueldos. Están en Contrato y AuditoriaContrato, y solo Recursos Humanos puede leerlas (RN21). Los demás ven el plantel por `vw_nomina`, que no tiene sueldos. La vista puede leer Contrato aunque el rol tenga DENY por el mismo encadenamiento de propiedad.
- Montos. Los gastos y el tope los ve solo Finanzas. Los demás roles no tienen permiso sobre Gasto ni sobre `vw_situacion_tope`.
- La auditoría de sueldos no la puede modificar nadie. La llena el trigger de Contrato, y Recursos Humanos solo la puede leer.
- Los km de las piezas y el cierre de Gran Premio. Nadie tiene UPDATE sobre Componente ni sobre GranPremio. Solo `usp_cerrar_gp` los cambia, así que el dato desnormalizado no se puede tocar a mano.

Cómo se probó

La sección 10 del script de pruebas se hace pasar por cada usuario con `execute as user = '...'` y vuelve con `revert`. Para cada rol prueba algo que tiene que poder hacer y algo que no:

Usuario	Prueba	Resultado
`usr_director`	INSERT directo en Resultado	Rechazado: The INSERT permission was denied on the object 'Resultado'
	El mismo resultado con `usp_cargar_resultado`	Se carga
	Leer Contrato	Rechazado
	Leer `vw_nomina`	Ve el plantel sin sueldos
	Marcar un Gran Premio como cerrado	Rechazado: The UPDATE permission was denied on the object 'GranPremio'
`usr_ingeniero`	Poner en 0 los km de una pieza	Rechazado
	Cargar un uso de pieza y cerrar Barcelona	Se cargan, y el cierre retira dos piezas
`usr_rrhh`	Cambiar un sueldo	Se cambia y la auditoría guarda `login_rrhh` como usuario
	Borrar la auditoría	Rechazado
`usr_finanzas`	INSERT directo en Gasto	Rechazado
	El mismo gasto con `usp_registrar_gasto`	Se carga
`usr_consulta`	Leer `vw_puntos_equipo_por_gp`	La ve
	Leer `vw_situacion_tope`	Rechazado

Centralización en la capa de datos

Todas las reglas de negocio viven en la base. Esta tabla dice para cada una qué parte de la base la hace cumplir y en qué sección del script de pruebas se ve funcionando.

Regla	Qué dice	Mecanismo	Prueba
RN01	Sesiones sprint solo en Grandes Premios sprint	Trigger `tr_Sesion_validar`	Sección 6
RN02	La sesión cae dentro del fin de semana	Trigger `tr_Sesion_validar`	Sección 6
RN03	Como mucho dos pilotos por Gran Premio	`usp_inscribir_piloto`	Sección 4
RN04	Solo pilotos con contrato vigente	`usp_inscribir_piloto` con `fn_contrato_vigente`	Sección 4
RN05	Un chasis no se repite en un Gran Premio	UNIQUE `UQ_Inscripcion_gp_chasis`	Sección 4
RN06	El chasis es de la temporada del Gran Premio	`usp_inscribir_piloto`	Sección 4
RN07	Sin contratos superpuestos	Trigger `tr_Contrato`	Sección 9
RN08	Titular o reserva solo en contratos de pilotos	Trigger `tr_Contrato`	Sección 9
RN09	Todo cambio de sueldo queda registrado	Trigger `tr_Contrato` y defaults de AuditoriaContrato	Secciones 9 y 10
RN10	El resultado es de un piloto inscripto en ese Gran Premio	`usp_cargar_resultado`	Sección 5
RN11	Posición solo si terminó, y sin repetirse	CHECK `CK_Resultado_posicion` y `usp_cargar_resultado`	Sección 5
RN12	Los puntos se calculan, no se guardan	Tabla Puntaje, `fn_puntos_piloto` y `vw_puntos_equipo_por_gp`	Secciones 1 y 2
RN13	Penalización por pasarse del límite de piezas	Trigger `tr_UsoComponente`	Sección 7
RN14	El cierre suma km y retira piezas gastadas	`usp_cerrar_gp` con el cursor	Secciones 3 y 10
RN15	Un Gran Premio se cierra una vez y cerrado no acepta usos	`usp_cerrar_gp` y trigger `tr_UsoComponente`	Secciones 3 y 7
RN16	Una pieza retirada o dañada no se usa más	Trigger `tr_UsoComponente`	Sección 7
RN17	Una pieza tiene como mucho un gasto de compra	Trigger `tr_Gasto_tope`	Sección 8
RN18	Para el tope solo cuentan ciertas categorías	`fn_gasto_computable`	Sección 8
RN19	No se puede pasar el tope ni gastar en un año sin tope	Trigger `tr_Gasto_tope`	Sección 8
RN20	El gasto que cruza el umbral deja una alerta	Trigger `tr_Gasto_tope`	Sección 8
RN21	Los sueldos solo los ve Recursos Humanos	DENY sobre Contrato y `vw_nomina`	Sección 10
RN22	Resultado, Inscripcion, Gasto y Componente solo por procedimiento	DENY sobre las tablas y EXECUTE sobre los procedimientos	Sección 10

Además de las reglas, en la base también quedan:

- Las validaciones de cada dato, con tipos, NOT NULL, CHECK, UNIQUE y claves foráneas.
- Los valores que no tiene que cargar nadie a mano, con defaults: la fecha y el usuario de la auditoría, el estado inicial de una pieza, el umbral del 90 %.
- Los cálculos que se usan en más de un lugar, en funciones: puntos, contrato vigente, km restantes, gasto computable.
- Las operaciones de varios pasos, en procedimientos con transacción.

- El manejo de errores y la seguridad.

¿Por qué en la base y no en una aplicación? Porque así valen para cualquiera que use los datos, por el camino que sea: SSMS, una aplicación, un script de carga o un usuario con permisos. Si el control del tope estuviera en una aplicación, alguien que cargara un gasto desde SSMS se lo saltearía. En la base está escrito una sola vez, y los permisos cierran los caminos que no pasan por los controles. Una aplicación que se haga después no tiene que repetir ninguna regla, solo llamar a los procedimientos y mostrar los mensajes.

9. Conclusiones

Las decisiones que más pesaron

Modelar una sola escudería y no el campeonato. Lo interesante del dominio es la gestión interna: las piezas, los contratos y el tope de gasto. Con los rivales adentro la base hubiera sido mucho más grande sin sumar ninguna regla nueva. El costo es que no hay tabla de posiciones, solo los puntos de nuestros pilotos y del equipo.

No guardar los puntos. Salen de cruzar cada resultado con la tabla Puntaje de esa temporada. Si cambia el reglamento se cambia una tabla y no hay que tocar ningún resultado.

Guardar los km de cada pieza aunque se puedan calcular. Es la única excepción a la 3FN y la hicimos a propósito, porque el cierre de Gran Premio decide con ese dato si retira una pieza. Para que no pueda quedar mal, solo lo cambia `usp_cerrar_gp` y ningún rol tiene permiso para tocarlo.

Colgar todo de la inscripción. Los resultados, los usos de piezas y las penalizaciones apuntan a Inscripcion y no al piloto. Así cada dato sabe de qué Gran Premio es y con qué chasis se corrió, sin repetir esas columnas.

Elegir el mecanismo más simple que alcanza. Si una regla se podía escribir como restricción, fue restricción (un CHECK o un UNIQUE), y los procedimientos no la vuelven a validar. Si tenía que valer siempre y miraba otras tablas, trigger. Si era una operación con varios pasos o con validaciones previas, procedimiento, y los permisos obligan a usarlo. El cursor quedó solo en el cierre de Gran Premio, que es el único lugar donde hace falta ejecutar algo por cada fila.

Armar los datos de prueba para las reglas y no al revés. Verstappen ya tiene un MGU-K de más, Hadjar está a uno del límite, 2026 va en 89,47 % del tope y una caja de cambios está a 300 km de su vida útil. Así cada regla se puede ver saltar con una o dos instrucciones.

Qué cambiaríamos

- El uso de piezas se registra por Gran Premio. Si se registrara por sesión se podría saber, por ejemplo, en qué sesión se rompió una pieza. Lo dejamos así porque el reglamento cuenta las piezas por temporada y el cierre suma los km una vez por Gran Premio.
- Los sueldos no cuentan para el tope. En la F1 real la mayoría sí cuenta (quedan afuera los pilotos y los tres sueldos más altos del equipo). Se podría agregar con una categoría de gasto para sueldos que se cargue todos los meses.
- Todos nuestros errores salen con el número 50000. Para el TP alcanza con el mensaje, pero si arriba de la base hubiera una aplicación, convendría que cada error tuviera su propio número para que la aplicación los distinga sin leer el texto.
- Todo está en dólares y no hay tipo de cambio. Muchos proveedores son europeos, así que en una base de verdad haría falta.

Qué aprendimos

Que un trigger se dispara una vez por instrucción, no una vez por fila. Hay que pensar `inserted` como una tabla. El caso más claro es el de las penalizaciones: si un mismo INSERT trae dos piezas de más, la primera tiene que llevar 10 puestos y la segunda 5, y eso solo sale bien si se cuentan todas juntas.

Que un CHECK no falla cuando la condición da desconocido. `posicion > 0` con la posición en NULL no da falso, da desconocido, y el CHECK lo deja pasar. Por eso `CK_Resultado_posicion` dice `posicion is not null` de forma explícita.

Que con el encadenamiento de propiedad un DENY sobre una tabla no impide escribir en ella a través de un procedimiento del mismo dueño. Con eso se arma todo el esquema de "solo por procedimiento" sin darle a nadie permisos sobre las tablas.

Que una transacción sirve si se piensa qué pasa cuando algo falla a la mitad. En la compra de una pieza, si el gasto se rechaza, la pieza que ya se había insertado tiene que desaparecer también.

Y que tener las reglas en la base y no en una aplicación cambia la forma de pensar el diseño. No alcanza con que el camino normal funcione: hay que pensar qué pasa si alguien entra por otro lado, y cerrar ese lado con un trigger o con un permiso.

Anexo. Los scripts

Están numerados en el orden en que se ejecutan.

Script	Qué hace
`01_estructura.sql`	Borra la base si existe, la crea de nuevo y arma las 25 tablas con sus restricciones
`02_datos.sql`	Carga los datos de prueba (temporadas 2025 y 2026)
`03_funciones.sql`	Las cinco funciones
`04_vistas.sql`	Las cuatro vistas
`05_procedimientos.sql`	Los seis procedimientos, con el cursor del cierre de Gran Premio
`06_triggers.sql`	Los cuatro triggers
`07_seguridad.sql`	Logins, usuarios, roles y permisos
`08_consultas.sql`	Las consultas y subconsultas del documento
`09_pruebas.sql`	Prueba cada regla de negocio, cada error y los permisos de cada rol
`TP_completo.sql`	Del 01 al 07 en un solo archivo

Cómo se corren

Lo más rápido es abrir `TP_completo.sql` en SSMS y apretar F5. Arma la base entera desde cero, y se puede correr las veces que haga falta porque empieza borrando la base.

También se pueden correr del 01 al 07 de a uno, en orden. Es lo que conviene mientras se trabaja en un área.

Después, con la base recién armada, `08_consultas.sql` muestra las consultas con los resultados que figuran en el documento. `09_pruebas.sql` va al final porque cambia los datos (cierra Grandes Premios, carga compras). Cada prueba avisa con un `print` qué tiene que pasar, y si algo se acepta cuando no debería, imprime `MAL`.

TP_completo.sql

Es el 01 al 07 pegados en orden. Solo lleva `GO` donde SQL Server lo obliga: después del `create database` y antes de cada función, vista, procedimiento o trigger. El `use EscuderiaF1` del principio de cada script se saca, porque a esa altura ya se está parado en la base.

No se edita a mano. Si se cambia alguno de los scripts del 01 al 07, se vuelve a armar pegándolos de nuevo, y se prueba corriéndolo dos veces seguidas.